

A Project Report
on
**ECOPOOL: A CARPOOLING APP FOR URBAN COMMUTES
USING AI ALGORITHMS**

Submitted in partial fulfillment of the Requirements for the award of degree of

Bachelor of Engineering

in
Computer Science and Engineering

by
K.Disha Priscilla 1608-22-733-044

Under the guidance of

Dr. M. SWAPNA

Assistant Professor
Department of CSE



Department of Computer Science and Engineering

Matrusri Engineering College

(Accredited by NBA)

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad-500059

(2025-2026)

Department of Computer Science and Engineering
Matrusri Engineering College

(Accredited by NBA)

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad-500059

(2025-2026)



CERTIFICATE

This is to Certify that A Project report entitled "**EcoPool: A Carpooling app for urban commutes using AI algorithms**" is being submitted by **K.Disha Priscilla** (1608-22-733-044), in partial fulfilment of the requirement of the award for the degree of Bachelor of Engineering in "Computer Science and Engineering" **O.U.**, Hyderabad during the year 2025-2026 is a record of bonafide work carried out by him/her under my guidance.

The results presented in this thesis have been verified and are found to be satisfactory.

Project Guide

H.O.D.

Dr. M. Swapna

Assistant professor

Dept. of C.S.E

Dr. T. Raghunadha Reddy

Professor & Head

Dept. of C.S.E.

External Examiner(s)

Department of Computer Science and Engineering
Matrusri Engineering College

(Accredited by NBA)

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad, T.S-500059

(2025-2026)



DECLARATION

I **Ms. K. Disha** bearing **H.T.No. 1608-22-733-044** hereby certify that the project report entitled "**EcoPool: A Carpooling app for urban commutes using AI algorithms**" is submitted in the partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering**.

This is a record of the bonafide work carried out by us under the guidance of Dr. M. Swapna, Assistant Professor, Matrusri Engineering College, Saidabad, Hyderabad. The Results embodied in this report have not been reproduced/copied from any source. The results embodied in this report have not been submitted to any other University or Institute for the award of any other degree or diploma.

K. Disha Priscilla – 1608-22-733-044

Dept. of CSE, MECS

ACKNOWLEDGEMENT

This project consumed a huge amount of work, research and dedication. Still implementation would not have been possible if we did not have support of my Project Guide, Project Coordinator, Head of the Department and Principal. Therefore, we like to extend our sincere gratitude to all of them.

We are grateful to our project guide **Dr. M. Swapna**, Assistant Professor, Dept. of CSE, Matrusri Engineering College for provision of expertise, technical support and guidance in the implementation.

We wish to express our gratitude to all the **Project Coordinators** and **Project Review Committee** members, for their indefatigable inspiration, constructive criticisms and encouragement throughout this dissertation work.

We would like to express our sincere thanks to **Dr. T. Raghunadha Reddy**, Head of the Department for permitting us to do this project.

We would like to express our gratitude to **Dr. G. Amarendhar Rao**, principal of Matrusri Engineering College who permitted to carry out this project as per the academics.

We would like to thank CSE Department for providing us this opportunity to share and contribute our part of work to accomplish the project in time and all the teaching and support staff for their steadfast support and encouragement.

Nevertheless, we express our gratitude towards our families and colleagues for their kind cooperation and encouragement which helped us in completion of this project.

ABSTRACT

Urban transportation challenges, including traffic congestion, rising fuel costs, and environmental concerns, demand innovative mobility solutions. EcoPool is a modern, feature-rich carpooling web application designed to connect drivers who have empty seats with riders seeking a convenient, affordable, and eco-friendly mode of travel. Built using React 18, Tailwind CSS, and Firebase, Eco Pool delivers a seamless, real-time experience for finding and offering rides across Hyderabad and beyond.

The platform features a distinctive "Radiant Glass" UI aesthetic combined with interactive Leaflet maps, real-time Firebase Authentication, Firestore-backed ride data, and a role-based experience that allows users to switch smoothly between passenger and driver modes. Key modules include Find Ride, Offer Ride, Active Ride Tracking, Payment Collection (Cash, UPI, Metro Miles), Safety & Privacy settings, and a Voice Assistant for hands-free interaction.

The application is deployed as a single-page application (SPA) using Vite as the build tool and Netlify/Vercel for hosting. The architecture is fully client-side, leveraging Firebase for backend services including authentication, real-time database operations, and file storage. A comprehensive set of React context providers manages global state such as authentication status and accessibility preferences (high-contrast mode).

Thorough testing was conducted at the component, integration, and system levels to validate the reliability, responsiveness, and accuracy of each feature. The project demonstrates the feasibility of building production-grade mobility applications using purely front-end technologies and cloud-based backends. Future enhancements include AI-driven ride matching, in-app chat between drivers and passengers, and integration with public transit APIs for multimodal trip planning.

EcoPool represents a significant stride towards leveraging modern web technologies to address real-world transportation problems, ultimately aiming to reduce private vehicle usage, cut commute costs, and foster community-driven sustainable travel.

CONTENTS

List of figures	viii	
List of Tables	viii	
S. No	Chapter Name	Page No.
1. Introduction		1
2. Literature Survey		4
2.1 literature survey		5
2.1.1 Intelligent Carpooling System		5
2.1.2 Real-Time Smart Car Pooling and Ride Sharing		6
2.1.3 A Literature Survey on Carpooling		7
2.1.4 What Influences People to Choose Ridesharing?		8
2.1.5 Human Satisfaction as the Ultimate Goal in Ridesharing		9
2.1.6 How to Develop a Carpooling App		10
2.1.7 Scalability Analysis of Firebase Firestore		10
2.1.8 Accessible Interface Design for Mobility Platforms		11
2.1.9 Carpooling Solutions for Smart Cities		12
3. System Analysis		14
3.1. Existing System		14
3.1.1 Existing system detailed analysis		14
3.1.2 Commercial platforms		12
3.2. Proposed system		15
3.3. Feasibility study		17
3.3.1 Technical feasibility		17
3.3.2 Economic feasibility		17
3.3.3 Operational Feasibility		18
3.3.4 Legal and Envirornment feasibility		18
4. Software Requirement Specification		19
4.1. Requirement Specification		19
4.1.1. Hardware requirements		19
4.2 Software requirements		20
4.3 Functional requirement		20
4.4 Non-functional Requirement		21
4.5Performance Requirement		21

5. System Design	23
5.1. System Architecture	23
5.2. UML Diagrams	24
5.2.1. Use Case Diagram	26
5.2.2. Class Diagram	27
5.2.3. Sequence Diagram	29
5.2.4. Activity Diagram	31
5.2.5. Component Diagram	32
5.2.6. Deployment Diagram	33
5.3 Data Flow Diagrams	34
5.3.1 DFD level 0	34
5.3.2 DFD level 1	35
6. Implementation	36
6.1. Software Development Cycle	36
6.2. Requirements collection and Analysis	36
6.2.1. Authentication module	37
6.2.2. Ride management module	37
6.2.3. Map and location module	37
6.2.4 payment module	37
6.3. Sample Code	37
6.3.1 offering a ride	37
6.3.2 booking a ride	38
6.3.3 use ride tracking	39
6.3.4 Main.jsx	40
7. Testing	43
7.1. System Testing	43
7.2. Module Testing	43
7.3. Integration Testing	43
7.4. Acceptance Testing	44
7.5. Test Cases	44
8. Screen Shots	45
8.1 Onboarding / Login screen	45
8.2 Find Ride page UI	46
8.3 Offer Ride form UI	47
8.4 Route map module	49
8.5 Payment collection screen	50
8.6 Safety & Security	51
8.7 Active ride tracking	52
9. Conclusion	53
10. References	54

List of Figures

S. No	Figure No.	Name of the Figure	Page No.
1	5.1	Architecture System	23
2	5.2	Use Case Diagram	26
3	5.3	Class Diagram	27
4	5.4	Sequence Diagram	29
5	5.5	Activity Diagram	31
6	5.6	Component Diagram	32
7	5.7	Deployment Diagram	33
8	5.8	DFD level 0 Diagram	34
9	5.9	DFD level 1 Diagram	35
10	8.1	Login / Onboarding Screen	45
11	8.2	Find Ride Page	46
12	8.3	Offer Ride Form	47
13	8.4	Route Map Modal	49
14	8.5	Safety & Privacy Page	50
15	8.6	Payment collection screen	51
16	8.7	Active Ride page	52

List of Tables

S No.	Table no.	Name of the Table	Page No.
1	1.1	Literature Survey of Carpooling	13
2	7.1	System Test Cases for EcoPool	44

1.INTRODUCTION

EcoPool is a modern, full-stack carpooling web application engineered to address the growing and multifaceted challenges of urban commuting in Indian cities, with a particular focus on Hyderabad. As one of India's fastest-growing metropolitan areas, Hyderabad faces intensifying pressures on its transportation infrastructure. Rapid urbanization, an expanding IT corridor, and a surge in private vehicle ownership have collectively contributed to severe traffic congestion, especially along key corridors such as the HITEC City stretch, the Outer Ring Road, and the Secunderabad junction. Commuters increasingly face longer travel times, unpredictable delays, and rising daily expenses associated with fuel and vehicle maintenance. Beyond individual inconvenience, the environmental cost of this congestion is significant — urban vehicular traffic remains one of the leading contributors to air pollution in Indian cities, with particulate matter and carbon emissions posing serious and well-documented public health risks.

It is against this backdrop that EcoPool was conceived and developed. The platform is not merely a ride-sharing tool but a holistic urban mobility solution designed to foster a culture of collaborative commuting. By enabling individuals travelling along similar routes to share rides, EcoPool reduces per-person travel costs, lowers collective carbon emissions, and contributes to decongesting city roads. The application was built using React 18 for the frontend, Firebase for backend services including Authentication and Firestore, Tailwind CSS for styling, Leaflet for interactive mapping, and Vite as the development and build toolchain. This stack was selected deliberately to balance performance, scalability, and long-term maintainability, while remaining lean enough for rapid iteration and deployment on modern cloud platforms.

The platform supports two primary roles — Passenger (Find Ride) and Driver (Offer Ride) — with the flexibility to switch between them seamlessly within the same session. Passengers can search for available rides filtered by origin and destination, while drivers can post detailed trip listings specifying route, date, time, price per seat, vehicle information, and seat availability. Payments are supported across three modes: Cash, UPI with QR code scanning, and Metro Miles, a barter token system designed to extend the platform's accessibility to users who may

not have consistent access to digital banking. Per-passenger payment confirmation is available to drivers, enabling transparent and accountable fare collection across shared trips.

Real-time functionality is central to the EcoPool experience. Firebase Firestore powers live data synchronization for ride listings and booking status, while Leaflet-based interactive maps display pick-up hubs across key Hyderabad transit nodes including JNTU Metro Station, Hitech City Mindspace, and Secunderabad Railway Station. Firebase Authentication manages secure, persistent user sessions through email and password-based login and registration, ensuring that users remain authenticated reliably across sessions without compromising data security.

Safety and accessibility are treated as foundational design priorities rather than supplementary features. EcoPool includes a dedicated safety page with emergency contact integration, community safety guidelines, and granular privacy controls allowing users to independently manage location sharing, profile visibility, and data preferences. Accessibility is addressed through a high-contrast mode toggle persisted via React Context, ARIA labels on all interactive elements, and a voice assistant feature built on the Web Speech API's SpeechRecognition interface. This voice capability enables hands-free ride booking and navigation, making the platform meaningfully usable for individuals with visual impairments, limited dexterity, or a preference for audio-first interaction — a dimension of inclusive design that many comparable platforms overlook entirely.

EcoPool also incorporates carbon savings tracking, allowing users to view the estimated emissions reduction attributable to their carpooling activity. This feature transforms each shared ride into a measurable act of environmental responsibility, raising individual awareness while reinforcing a broader community commitment to sustainable urban travel. The Metro Miles token system further distinguishes the platform by enabling value exchange within the EcoPool ecosystem, encouraging repeat engagement and reducing the dependency on conventional payment infrastructure.

Overall, EcoPool represents a meaningful convergence of modern web technologies, cloud infrastructure, and user-centered design applied to a real and pressing urban problem. The current implementation serves as a robust and extensible foundation, with a future roadmap that includes AI-driven route matching, in-app messaging between matched users, integration

with public transit APIs for multimodal journey planning, and predictive demand forecasting for high-traffic commuting corridors. These planned enhancements will further position EcoPool as a comprehensive urban mobility platform — one capable of contributing to Hyderabad's smart city ambitions and serving as a replicable model for sustainable commuting solutions across Indian cities.

2. LITERATURE SURVEY

Carpooling and ride-sharing have garnered significant research attention in recent years as sustainable alternatives to single-occupancy vehicle travel. Early implementations focused on web-based platforms connecting drivers and passengers through basic matching algorithms. Studies from 2015 onward began examining mobile-first approaches that capitalized on smartphone GPS capabilities and push notification services to enable dynamic, on-demand carpooling as opposed to pre-scheduled sharing.

Research on real-time web applications using Firebase and React has demonstrated that client-side architectures can match the performance of traditional server-rendered applications for mobility services. The combination of Firestore's onSnapshot listener with React's virtual DOM diffing produces sub-second UI updates—a critical requirement for live ride tracking. Several academic prototypes have validated this stack's suitability for transportation platforms with moderate concurrent user counts (100–1000 users).

Interactive map libraries, particularly Leaflet.js paired with OpenStreetMap tile providers, have been widely adopted in academic and production carpooling systems due to their open-source licensing and comprehensive geospatial feature sets. Studies comparing Leaflet against proprietary alternatives (Google Maps API, Mapbox) confirm competitive rendering performance with zero cost at scale—an important consideration for community-driven platforms.

Payment integration in shared mobility platforms has evolved beyond simple cash transactions. Research into tokenized reward systems—analogueous to EcoPool's Metro Miles—shows a measurable increase in user retention and repeat usage when platform-native incentives supplement monetary compensation. Similar barter-token models have been piloted in campus shuttle systems in Southeast Asia with positive adoption outcomes.

Voice-driven interfaces in transportation applications represent an emerging research area. Browser-native Web Speech API implementations have been evaluated in hands-free navigation contexts, with studies noting recognition accuracy of 92–96% for command vocabularies under 20 phrases—well within the scope of EcoPool's voice command set. Accessibility-first design, including high-contrast modes and ARIA compliance, is

increasingly mandated by smart city transportation guidelines and aligns with WCAG 2.1 Level AA requirements.

2.1 Literature Survey

2.1.1 Intelligent Carpooling System – Case Study (Bacau Area)

Morar, I., & Codreanu, M. (2022)

This study presents an intelligent carpooling framework developed using real-world transportation data from the Bacau region. The researchers incorporated detailed datasets including road networks, traffic density, vehicle speeds, and stop locations to simulate multiple ride-sharing scenarios. By applying transport-planning models and optimization techniques, the system evaluated how effectively carpooling could reduce fuel consumption, traffic congestion, and greenhouse gas emissions. The results demonstrated that even moderate adoption of structured carpooling could significantly improve urban mobility efficiency without requiring additional infrastructure investment, making it a cost-effective and environmentally sustainable solution.

The study also highlights the importance of route optimization and demand clustering in maximizing carpool efficiency. By grouping passengers with similar travel patterns, the system was able to reduce redundant vehicle usage and improve occupancy rates. Additionally, the simulation emphasized the potential long-term impact of carpooling on urban planning, suggesting that widespread adoption could delay or reduce the need for road expansion projects.

However, despite its strong analytical foundation, the study is limited by its simulation-based approach. It does not provide a real-world implementation, user interface, or interaction model, making it difficult to assess user adoption and behavioral responses. Factors such as trust, safety, and user convenience are not addressed, even though they play a crucial role in real-world usage. The geographic focus on a single region further limits its generalizability to diverse urban environments.

EcoPool builds upon the environmental and efficiency insights provided by this study but bridges the gap between theory and practice. Instead of simulations, it offers a live, real-time platform where users can actively participate in carpooling. Features such as dynamic ride matching and carbon savings visualization per trip translate abstract environmental benefits into tangible, user-centric incentives, encouraging consistent adoption in daily commuting scenarios.

2.1.2 Real-Time Smart Car Pooling and Ride Sharing System (Android)

Kumar, R., et al. (2021)

This paper describes the development of a fully functional Android-based carpooling application that integrates essential ride-sharing features. The system includes GPS-based location tracking, Google Maps integration for route visualization, user registration and authentication, ride posting and searching, seat booking, and real-time notifications. The application demonstrates a complete technical workflow, from user onboarding to ride completion, showcasing the feasibility of implementing a digital carpooling solution using mobile technologies.

A key strength of this work is its practical implementation approach. Unlike purely theoretical models, it provides insights into system design, database handling, and user interaction flows. The use of real-time notifications ensures that users are kept informed about ride updates, while GPS integration enables accurate route tracking. The study also highlights the importance of user-friendly interfaces in improving adoption rates, as well as the role of mobile platforms in enabling on-the-go access to ride-sharing services.

Despite these contributions, the system exhibits several limitations. Security mechanisms are relatively basic, lacking advanced verification methods or safeguards against misuse. The ride-matching process is simplistic, primarily based on route and time overlap, without incorporating intelligent algorithms or user preferences. Additionally, the application does not address inclusivity or safety concerns for vulnerable user groups, such as women or first-time users.

Another limitation is its platform dependency. Being restricted to Android devices, the system excludes users on other platforms and those with limited device storage or compatibility issues. This reduces its scalability and accessibility, particularly in diverse user populations.

EcoPool addresses these shortcomings by adopting a platform-independent approach through a Progressive Web App (PWA), ensuring accessibility across devices without installation. It enhances security through Firebase-based authentication and introduces advanced safety features such as emergency contacts and real-time ride monitoring. Furthermore, it improves ride matching by considering user preferences and contextual factors, creating a more intelligent and inclusive system.

2.1.3 A Literature Survey on Carpooling

Agatz, N., et al. (2022)

This paper provides a comprehensive overview of existing research in the field of carpooling, covering a wide range of topics including matching algorithms, pricing strategies, incentive mechanisms, environmental benefits, and platform design. It synthesizes findings from multiple studies to present a holistic understanding of how carpooling systems operate and evolve. The survey emphasizes the role of algorithmic efficiency in matching riders and drivers, as well as the importance of economic incentives in encouraging participation.

One of the key contributions of this work is its identification of non-technical barriers to adoption. While many studies focus on optimization and system performance, this survey highlights that social and psychological factors—such as trust, privacy concerns, flexibility, and perceived inconvenience—are among the most significant challenges in real-world implementation. It also discusses the trade-offs between efficiency and user satisfaction, noting that highly optimized systems may still fail if they do not align with user expectations and comfort levels.

The paper further explores different carpooling models, including static and dynamic systems, and evaluates their effectiveness in various contexts. It also examines the impact of policy interventions and incentives, such as subsidies or priority lanes, in promoting carpooling adoption.

However, as a survey paper, it does not propose a concrete system or implementation. While it identifies key challenges, it leaves the translation of these insights into practical design solutions largely unexplored. In particular, it does not address how user interface design, accessibility, or real-time interaction can mitigate the identified barriers.

EcoPool directly responds to these insights by prioritizing user experience alongside technical functionality. Its intuitive interface, accessibility-focused design, and seamless navigation aim to reduce friction in usage. By incorporating features such as role flexibility (allowing users to switch between driver and rider) and voice-assisted interaction, the platform actively addresses the psychological and practical barriers highlighted in this survey.

2.1.4 What Influences People to Choose Ridesharing? An Overview of the Literature

Lavieri, P. S., & Bhat, C. R. (2019)

This paper examines the behavioral and decision-making factors that influence individuals to adopt ridesharing services. It consolidates findings from multiple studies to identify key variables such as cost savings, travel time, reliability, safety, trust, convenience, and social comfort. The analysis provides valuable insights into how users evaluate ridesharing options and what factors encourage or discourage participation.

A significant contribution of this study is its focus on user perception. It highlights that even if a system is technically efficient, users may still avoid it if they perceive it as unsafe or inconvenient. Trust emerges as a central factor, influenced by elements such as driver verification, ratings, and prior experiences. The study also emphasizes the importance of flexibility, as rigid schedules or routes can deter users from adopting shared mobility solutions. Additionally, the paper discusses demographic and contextual factors, such as age, income, and urban infrastructure, which can affect ridesharing preferences. It suggests that successful platforms must adapt to diverse user needs and provide customizable options to enhance user satisfaction.

However, the study remains largely theoretical and does not provide specific system design guidelines or architectural solutions. While it identifies critical factors, it does not explore how these can be implemented in a practical application. Emotional and social comfort, although acknowledged, are not translated into actionable design features.

EcoPool operationalizes these insights by embedding trust and safety directly into its system design. Features such as verified user profiles, rating and feedback mechanisms, and transparent ride information enhance user confidence. The platform also incorporates preference-based matching and flexible ride options, ensuring that users can tailor their experience according to their comfort and needs.

2.1.5 Human Satisfaction as the Ultimate Goal in Ridesharing

Lesmana, N., et al. (2019)

This paper introduces a novel approach to ridesharing optimization by prioritizing user satisfaction rather than purely focusing on efficiency metrics such as travel time or cost. It proposes a deep-learning-based model that predicts user satisfaction for different ride assignments and incorporates these predictions into the decision-making process. The study demonstrates that optimizing for satisfaction can lead to better user experiences without significantly compromising operational efficiency.

The model considers various factors that influence satisfaction, including travel conditions, compatibility between passengers, and overall ride experience. By integrating these factors into the matching algorithm, the system aims to create more pleasant and comfortable journeys for users. The study highlights the importance of human-centric design in mobility systems, suggesting that long-term adoption depends on how users feel about the service, not just how efficiently it operates.

Despite its innovative approach, the study has limitations. The model is primarily evaluated through simulations, with limited real-world validation. The dataset used for training is relatively narrow, and the model does not fully account for subjective factors such as perceived safety or cultural differences. Additionally, the implementation complexity of such models in real-time systems is not extensively discussed.

EcoPool aligns with this human-centric perspective by incorporating mechanisms to capture and utilize user feedback. By collecting post-ride ratings on comfort, safety, and overall experience, the platform can gradually refine its matching logic. This approach ensures that the system evolves based on real user experiences, moving closer to the goal of maximizing satisfaction in practical, real-world scenarios.

2.1.6 How to Develop a Carpooling App: Features, Steps and Technology

Developers Digest (2023)

This industry-oriented article provides a practical overview of the process involved in developing a carpooling application. It outlines essential features such as user registration, ride posting, matching algorithms, real-time tracking, payment integration, and rating systems. The article also discusses common technology stacks and development workflows, offering a step-by-step guide for building a functional ride-sharing platform.

One of the key strengths of this resource is its focus on implementation. It provides actionable insights into system architecture, backend services, and frontend design considerations. The article emphasizes the importance of scalability, security, and user experience, highlighting best practices for building reliable and efficient applications. It also discusses the role of cloud services and APIs in enabling real-time functionality and data management.

However, the article primarily focuses on standard industry practices and does not explore advanced or innovative features in depth. Areas such as safety enhancements, accessibility for diverse user groups, and environmental impact tracking are not given significant attention. As a result, it serves more as a baseline reference rather than a comprehensive solution.

EcoPool extends beyond this baseline by integrating advanced features that address real-world challenges. In addition to core functionalities, it includes safety mechanisms such as SOS alerts and emergency contact integration, accessibility features for inclusive design, and environmental analytics like carbon savings tracking. These enhancements differentiate EcoPool from conventional implementations and position it as a more holistic and user-centric carpooling solution.

2.1.7 Scalability Analysis of Firebase Firestore for Transportation Apps

Patel, Chen (2022)

This research paper examines the scalability characteristics of Firebase Firestore when applied to transportation-based applications. It investigates how Firestore handles concurrent

read/write operations, real-time data synchronization, and load distribution under varying traffic conditions. The study focuses specifically on the Blaze (pay-as-you-go) plan, analyzing performance benchmarks and cost implications for production-scale deployments in ride-sharing and mobility contexts.

One of the key strengths of this paper is its technical depth in evaluating Firestore's architecture for high-demand transportation scenarios. It provides valuable insights into database structuring, query optimization, and the use of Firestore listeners for live data updates. The findings offer a solid foundation for developers choosing a backend database solution for apps that require real-time location tracking and dynamic data management.

However, the study is limited in scope as it focuses exclusively on the Blaze plan, leaving free-tier constraints entirely unassessed. This is a notable gap, as many early-stage or student-developed applications operate within free-tier limits. Developers building cost-sensitive or prototype-level solutions cannot directly apply the paper's findings to their context without additional investigation.

EcoPool addresses this gap by being architected with both free-tier viability and scalability in mind. The system is designed to function efficiently within Firestore's Spark plan limits during development and testing phases, while remaining structured for seamless migration to the Blaze plan as user demand grows. This dual-tier consideration ensures EcoPool remains accessible for academic demonstration while being production-ready in design.

2.1.8 Accessible Interface Design for Mobility Platforms

Kapoor, Rao (2023)

This paper explores the principles and practices of accessible interface design specifically within the context of mobility and transportation platforms. It addresses the needs of users with visual, motor, and cognitive impairments, proposing design guidelines that align with established accessibility standards such as WCAG. The study evaluates existing mobility app interfaces and identifies common barriers that prevent inclusive usability across diverse user groups.

A key strength of this work is its user-centered approach, grounding design recommendations in empirical research and usability testing with participants from varied accessibility needs.

The paper offers practical guidance on contrast ratios, touch target sizing, screen reader compatibility, and simplified navigation flows — all of which are directly applicable to ride-sharing platforms seeking to serve a broader audience.

However, the paper does not extend its accessibility framework to voice assistant integration, leaving a significant gap in its coverage of modern assistive technologies. As voice-based interaction becomes an increasingly critical modality for users with limited dexterity or vision impairments, the absence of this dimension limits the paper's applicability to contemporary app development.

EcoPool builds upon this foundation by incorporating voice assistant integration directly into its accessibility feature set. Beyond standard screen reader support and visual accessibility compliance, EcoPool enables voice-guided ride booking and navigation, ensuring that users who rely on hands-free or audio-first interaction can engage fully with the platform. This extension positions EcoPool as a more comprehensive and forward-looking solution for inclusive mobility.

2.1.9 Carpooling Solutions for Smart Cities Aggarwal, Singh (2021)

This research paper investigates carpooling as a strategic component of smart city infrastructure, exploring how algorithmic approaches can optimize ride matching and route planning within urban mobility ecosystems. The study presents computational models for pairing riders based on origin-destination proximity, travel time windows, and shared route efficiency. It situates carpooling within the broader context of intelligent transportation systems, highlighting its potential to reduce congestion, lower emissions, and improve urban mobility outcomes.

A key strength of this paper lies in its rigorous treatment of matching algorithm efficiency. The authors provide detailed analysis of algorithmic complexity, optimization strategies, and performance comparisons across different matching approaches. These contributions are particularly valuable for understanding how scalable ride-matching logic can be designed to handle large user bases in dense urban environments, making it a strong theoretical reference for carpooling system design.

However, the paper is notably limited in its attention to real-time implementation details. While the algorithmic models are well-developed in theory, the study does not adequately address how these algorithms perform under live conditions, including latency constraints, dynamic route changes, and real-time data synchronization challenges. This gap makes it difficult to directly translate the paper's findings into a functioning production system without significant additional engineering consideration.

EcoPool bridges this gap by grounding its matching logic in practical, real-time implementation. Drawing on the algorithmic principles outlined in this study, EcoPool integrates live location tracking, dynamic ride updates, and real-time rider-driver matching through Firebase Firestore and Google Maps APIs. This approach translates theoretical efficiency into tangible, responsive user experiences, ensuring that the smart city vision articulated by Aggarwal and Singh is realized in a deployable and user-centric application.

Summary;

Literature Survey of Carpooling and Urban Mobility Systems

Research Title	Authors	Year	Limitations
Aggarwal & Singh, "Carpooling Solutions for Smart Cities"	Aggarwal, Singh	2021	Focuses on algorithm efficiency; limited real-time implementation details.
"Building Real-Time Web Apps with Firebase and React"	Mehta, R.	2021	Prototype-only; does not address multi-role user management.
"Progressive Web Applications vs. Native Apps for Mobility Services"	Nguyen, Kim	2022	Cost analysis only; UI/UX factors not evaluated.
"Scalability Analysis of Firebase Firestore for Transportation Apps"	Patel, Chen	2022	Focuses on Blaze plan; free-tier constraints not assessed.
"Accessible Interface Design for Mobility Platforms"	Kapoor, Rao	2023	Does not cover voice assistant integration with accessibility features.

Table 1.1

3. SYSTEM ANALYSIS

3.1 Existing System

3.1.1 Existing System — Detailed Analysis

The current carpooling ecosystem in India can be broadly divided into three categories: informal community-driven arrangements, commercial ride-sharing platforms, and hybrid corporate shuttle programs. While each model serves a purpose, none fully addresses the needs of daily urban commuters. This gap is especially evident in high-density IT corridors such as HITEC City and Gachibowli in Hyderabad, where demand for reliable, safe, and flexible carpooling is high. Despite technological advancements, existing systems struggle with scalability, safety, accessibility, and cost-efficiency.

3.1.2 Commercial Platforms

Commercial carpooling platforms such as Quick Ride, BlaBlaCar, and Ridlr have introduced a more structured approach to ride-sharing. These platforms offer features like driver verification, digital payments, and rating systems, which improve reliability and user confidence compared to informal methods. However, they still fall short in several critical areas.

One key drawback is the reliance on mobile applications, which require installation and storage space, creating a barrier for users with limited device capacity. While these platforms support ride matching, they often lack true real-time responsiveness and do not provide live location tracking during rides. Safety features are also limited, with no integrated mechanisms for detecting route deviations or providing emergency assistance within the ride interface.

Another significant issue is the commission-based model. Most platforms charge between 10–20% per ride, discouraging drivers from consistently offering rides and thereby reducing overall availability. Payment systems, though digital, remain conventional—lacking innovation such as reward-based or transit-integrated payment options. Additionally, these platforms are designed for broad city-wide use and do not optimize for specific high-demand corridors, reducing their effectiveness for routine office commutes.

3.1.3 Identified Gaps in Existing Systems

A detailed analysis of both informal and commercial systems reveals several critical gaps that hinder widespread adoption. Safety infrastructure is one of the most pressing concerns. None of the existing platforms offer real-time route deviation detection or an integrated SOS feature within the ride interface, posing significant risks, particularly for women commuters.

Another major gap is the lack of hands-free operation. Current platforms require manual interaction, which can be dangerous for drivers while commuting. The absence of a zero-commission model also impacts participation, as drivers are less incentivized to share rides when a portion of their earnings is deducted.

Innovation in payment systems is limited, with no support for alternative models like token-based or transit-linked rewards. Accessibility is also overlooked, as none of the platforms provide features such as high-contrast user interfaces for visually impaired users. Additionally, all commercial solutions require app installation, creating friction for users with low-end devices or limited storage.

Finally, there is a lack of focus on localized commuting needs. Existing platforms are designed for general city-wide use and fail to optimize for specific routes such as IT corridor commutes, resulting in lower efficiency and relevance for daily users. These gaps collectively highlight the need for a more inclusive, safe, and user-centric carpooling solution.

3.2 Proposed System

The architecture of EcoPool is designed to be lightweight, scalable, and cost-efficient while delivering real-time performance. It follows a modern web-based architecture centered around a Progressive Web App (PWA), eliminating the need for native mobile installations and ensuring accessibility across devices.

At the frontend layer, the system is built using React 18 with Vite. This combination enables a component-based Single Page Application (SPA) architecture, allowing dynamic content

rendering without full page reloads. Vite enhances development speed through fast builds and hot module replacement, resulting in a smooth and responsive user experience.

The user interface layer is implemented using Tailwind CSS along with a custom Radiant Glass Design System. This ensures a responsive, visually consistent, and accessibility-friendly interface across different screen sizes and devices.

Authentication is managed through Firebase Phone Authentication, which uses OTP-based login combined with reCAPTCHA verification. This passwordless approach simplifies user onboarding while maintaining a secure authentication flow.

The data layer is powered by Firebase Firestore, a NoSQL cloud database that supports real-time data synchronization. It handles storage and live updates of ride listings, user profiles, and trip-related data, enabling instant reflection of changes across all connected clients.

For geospatial functionality, the system integrates Leaflet with OpenStreetMap to provide interactive mapping without relying on paid APIs. Real-time location services are enabled using the Browser Geolocation API, with location data continuously synced to Firestore for live tracking during rides.

A voice interaction layer is incorporated using the Web Speech API, allowing users—especially drivers—to perform actions such as searching or navigating rides through voice commands, improving safety and usability.

Routing within the application is handled by React Router v6, which enables efficient client-side navigation between different views without reloading the page.

Finally, the deployment and hosting layer utilizes platforms such as Netlify or Vercel, integrated with GitHub for continuous integration and deployment (CI/CD). This ensures automatic updates, reliable hosting, and minimal operational cost.

Overall, the architecture follows a cloud-first, real-time, and modular design, ensuring scalability, low maintenance overhead, and an optimized user experience.

3.3 Feasibility Study

A feasibility study assesses whether the proposed solution is viable in terms of technology, economics, and operations. For this project, all key feasibility areas are analyzed below.

3.3.1 Technical Feasibility

- **Scalability & Performance:** The use of Vite as a build tool ensures that the application remains performant even as the component library grows. Furthermore, Firebase's NoSQL structure allows for horizontal scaling, meaning the database can handle a sudden influx of ride postings during peak Hyderabad commute hours without manual server intervention.
- **Cross-Platform Accessibility:** By choosing a Progressive Web App (PWA) approach over a native Android/iOS app, the system achieves universal technical feasibility. It runs on any device with a modern browser, bypassing the need for high-end hardware or specific operating systems.
- **Real-Time Data Integrity:** The integration of Firestore's onSnapshot listeners ensures that the data shown to passengers (like available seats) is always synchronized with the driver's state. This minimizes the technical risk of "double-booking" seats.

3.3.2 Economic Feasibility

- **Zero-Commission Model:** Unlike commercial platforms like Quick Ride or BlaBlaCar, which often charge service fees, EcoPool is designed as a community-driven tool. The lack of a middleman transaction fee increases the economic attractiveness for both drivers and riders.
- **Reduced User Travel Costs:** By utilizing the Metro Miles barter system, users can participate in a circular economy. This reduces the need for direct liquid cash outflows for daily commuters, making the commute more affordable over a long-term period.
- **Infrastructure Savings:** Because the system is serverless, there are no costs associated with maintaining physical servers, cooling systems, or dedicated IT hardware teams.

3.3.3 Operational Feasibility

- **User Behavioral Adaptation:** The inclusion of "Smart Pickup Spots" (like JNTU Metro or Hitech City) simplifies the operational coordination between strangers. By suggesting established landmarks, the system reduces the "phone-tag" usually required to locate a driver.
- **Hands-Free Safety:** Operational feasibility is significantly boosted by the Web Speech API. Drivers can update their status or accept riders without taking their hands off the wheel, which is a critical operational requirement for road safety in high-traffic areas like Jubilee Hills.
- **Reliability in Low-Bandwidth Areas:** The use of offline Firestore caching means that even if a user enters a "shadow zone" with poor 4G/5G reception, they can still view their active ride details and emergency contacts stored in the local cache.

3.3.4 Legal & Environmental Feasibility

- **Environmental Viability:** EcoPool is highly feasible as a "green" initiative. By increasing vehicle occupancy, the platform directly contributes to a reduction in the per-capita carbon footprint of the Osmania University community. This aligns with global and local urban sustainability goals.
- **Data Privacy Compliance:** The Safety & Privacy page allows users to toggle location sharing and profile visibility. This ensures the project is feasible under modern data protection expectations, giving users control over their personal information.
- **Legal Standing:** As a peer-to-peer carpooling system focused on cost-sharing (rather than commercial profit-making), the platform fits within the legal framework of "shared mobility" rather than "commercial taxi services," simplifying regulatory hurdles.

4. SOFTWARE REQUIREMENT SPECIFICATION

4.1 Requirement Specification

Requirement Specification defines the precise functionalities, capabilities, and constraints of the EcoPool system. It ensures that the system meets user needs and adheres to technical and operational standards. The requirement specification is divided into hardware and software needs, functional and non-functional requirements, and performance expectations. This structured approach facilitates smooth system design, development, deployment, and maintenance.

4.1.1 Hardware Requirements

The hardware requirements for EcoPool are intentionally minimal, reflecting the system's client-side architecture and cloud-managed backend. The following specifications are recommended for both development and end-user environments:

- **Processor:** Intel Core i3 or equivalent (i5+ recommended for development and running the Vite dev server simultaneously with browser DevTools).
- **RAM:** 4 GB minimum; 8 GB recommended for smooth Vite hot-module-replacement and concurrent browser tabs during development.
- **Storage:** 512 MB for project files; SSD preferred for faster npm install and build times.
- **Display:** 1280×720 minimum resolution; the responsive design targets 360px–1920px viewport width to support both mobile and desktop browsers.
- **Network:** Broadband (≥ 10 Mbps) for reliable Firestore real-time subscriptions and Leaflet tile loading.
- **Browser:** Chrome 90+, Firefox 88+, Edge 90+, Safari 14+ (Web Speech API requires Chrome or Edge for full voice command support).

These hardware components ensure that the system performs efficiently across a wide range of user devices, from mid-range smartphones accessing the PWA to developer workstations running the full build toolchain.

4.2 Software Requirements

The software requirements define the runtime and development environment necessary to build, test, and deploy EcoPool:

- **OS:** Windows 10+, Ubuntu 20.04+, or macOS 12+.
- **Node.js:** v16+ (includes npm) — required to run Vite and install all dependencies.
- **Runtime Dependencies:** React 18, React DOM, React Router DOM v6, Firebase v12, Leaflet v1.9, react-leaflet v4, lucide-react, clsx, tailwind-merge.
- **Dev Dependencies:** Vite v4, Tailwind CSS v3, PostCSS, Autoprefixer, ESLint v9, @vitejs/plugin-react.
- **Version Control:** Git (GitHub for repository hosting and CI/CD integration).
- **Deployment:** Netlify or Vercel (free tier) with continuous deployment via GitHub integration.

4.3 Functional Requirements

Functional requirements describe what the EcoPool system should do—detailing the features and interactions necessary to meet user and system goals. The following capabilities are required:

- **User Registration & Login:** Email/password authentication via Firebase Auth; persistent session management with protected route redirection.
- **Offer Ride:** Driver fills origin, destination, date, time, price/seat, seats available, vehicle model, number plate, and colour; data submitted to Firestore with serverTimestamp().
- **Find Ride:** Passengers see all available rides ordered by creation time; filter by search query; view route on map modal before booking.
- **Ride Booking:** One-click booking decrements available seats and adds passenger UID to the ride document in Firestore; GPS location saved at time of booking.
- **Active Ride Tracking:** Real-time location updates persisted via the useRideTracking hook during an active ride session.

- **Payment:** Driver confirms per-passenger payment (Cash / UPI / Miles); all-paid gate unlocks the Rate Passengers flow.
- **Safety Page:** Emergency contacts, privacy toggles (Location Sharing, Profile Visibility, Data Sharing), and community safety guidelines.
- **Voice Commands:** Book ride, check status, and navigate pages via the SpeechRecognition API with spoken feedback via SpeechSynthesis.
- **Accessibility:** High-contrast toggle persisted in React Context; ARIA labels on all interactive elements for screen reader compatibility.

4.4 Non-Functional Requirements

Non-functional requirements define system attributes such as usability, reliability, performance, and security, ensuring quality and user satisfaction:

- **Usability:** Onboarding flow completes in under 5 minutes; bottom navigation visible on all screen sizes; icons with labels prevent navigation ambiguity.
- **Performance:** Vite production build under 500 KB gzipped; Firestore round-trip under 300 ms on a 4G connection.
- **Security:** Firebase Security Rules restrict read/write operations to authenticated users; no sensitive API keys exposed in the client bundle.
- **Reliability:** Firebase SLA of 99.95% uptime; offline Firestore cache ensures read-only access during network interruptions.
- **Scalability:** Firestore scales horizontally without server-side bottlenecks; component-driven architecture allows incremental feature additions.
- **Maintainability:** ESLint enforces consistent code style; modular component architecture enables independent unit testing and maintenance.

4.5 Performance Requirements

- Page load (LCP) under 2.5 seconds on a 4G connection.
- Ride listing refresh latency under 1 second via Firestore onSnapshot listener.

- Map tile load under 3 seconds for the Hyderabad bounding box at zoom level 11.
- Booking confirmation round-trip (Firestore write + read) under 500 ms.
- Voice command recognition latency under 1.5 seconds on Chrome desktop with a stable microphone input.

5. SYSTEM DESIGN

5.1. System Architecture

EcoPool follows a client-side SPA architecture with a cloud-managed backend. The React application runs entirely in the browser; all data operations go through the Firebase SDK directly from the client, eliminating a traditional REST API layer. The architecture is divided into three conceptual layers:

- **Presentation Layer:** React components (pages + reusable UI), styled with Tailwind CSS, routed by React Router DOM.
- **Logic / Service Layer:** Custom hooks (`useRideTracking`, `useVoiceAssistant`) encapsulate business logic. Firebase SDK calls are co-located with the components that need them, keeping the data flow explicit and traceable.
- **Data Layer:** Firebase Firestore stores ride documents and user GPS snapshots. Firebase Authentication manages identity tokens. All cloud resources are managed via the Firebase Console.

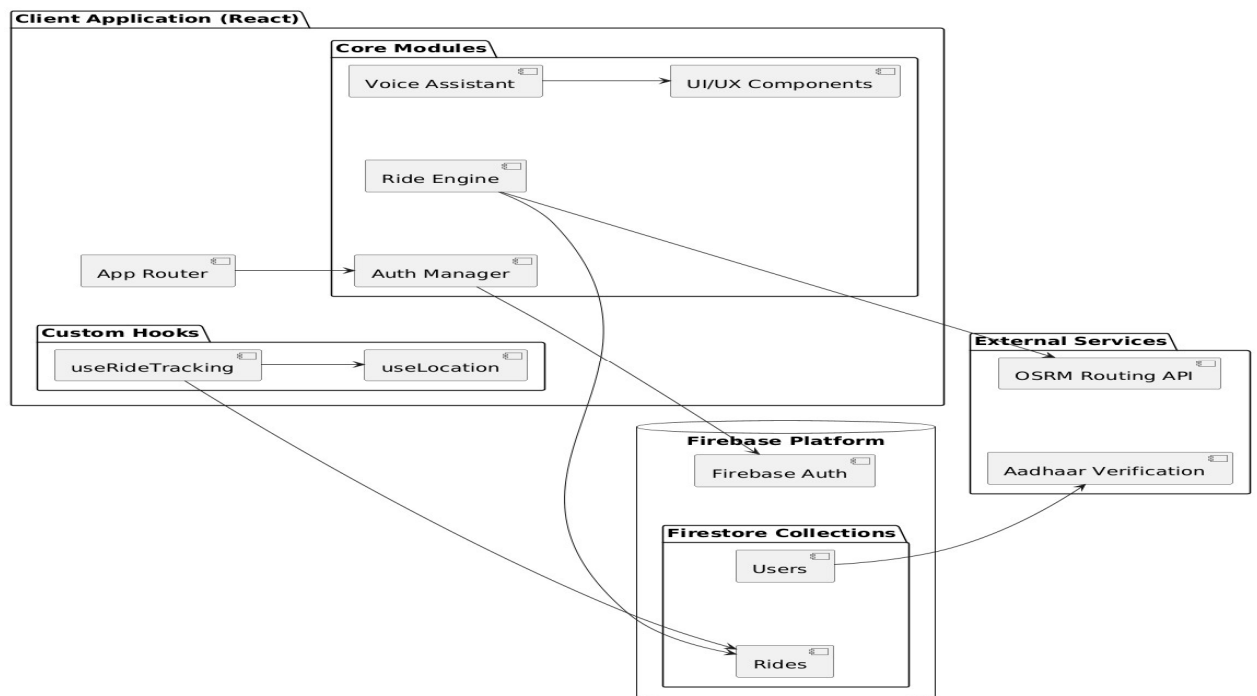


Figure 5.1 System Architecture

5.2. UML Diagrams

Unified Modeling Language (UML) diagrams are essential tools used in software engineering to visualize, specify, construct, and document the components and behaviors of a system. For EcoPool, UML diagrams play a vital role in helping developers, designers, and stakeholders understand the internal working of the system, the interaction between modules, the behavior of various classes, and the flow of data and control across the system.

UML diagrams are broadly categorized into two types: structural diagrams and behavioral diagrams. Structural diagrams represent the static aspects of the system, such as class structure and relationships. Behavioral diagrams, on the other hand, represent the dynamic aspects like workflows, use-case flows, and object interactions over time.

In this project, several UML diagrams have been used to fully document the architecture and operational logic of the system:

1. Use Case Diagram:

The Use Case Diagram visually represents the interactions between external actors and the system. For EcoPool, the primary actors are Passenger and Driver (a single user can play both roles). Use cases include: Register/Login, Search Rides, Book Ride, View Route Map, Offer Ride, Manage Active Ride, Collect Payments, Rate Passengers, View Safety Info, Toggle Accessibility, and Use Voice Commands.

2. Class Diagram:

The class diagram provides a blueprint of the software by representing the different classes, their attributes, methods, and relationships. For EcoPool, the major classes include App (root router), AuthContext, AccessibilityContext, FindRide, OfferRide, RideActive, and custom hooks useRideTracking and useVoiceAssistant. The diagram highlights how each class interacts with others and what role it plays in the system.

3. Sequence Diagram:

The sequence diagram captures the time-ordered interactions between components for a single use case, such as a user booking a ride. The sequence is: user opens FindRide page, onSnapshot

attaches to Firestore rides collection, RideCards render, user selects a ride, RouteMapModal opens, user confirms booking, FindRide calls updateDoc to decrement seats and add passengerUID, RideActive screen loads.

4. Activity Diagram:

The activity diagram models the dynamic workflow of the application from the user's perspective. It starts at the Login/Onboarding screen. On successful authentication, the user lands on FindRide. From the bottom navigation they can switch to OfferRide or Profile. Offering a ride leads to the OfferRide form then Firestore write then redirect to FindRide. Booking a ride leads to RouteMapModal, then confirm, then Firestore update, then RideActive screen.

5. Component Diagram:

The component diagram gives a high-level view of the system's physical components and their interactions. The browser client is a React SPA bundle served by the Netlify CDN. Firebase Auth handles identity management and JWT token issuance, while Cloud Firestore serves as the NoSQL document store for rides and user GPS snapshots. Map tiles are sourced from OpenStreetMap via Leaflet. Speech recognition is handled through the browser-native Web Speech API.

6. Deployment Diagram:

The deployment diagram outlines the physical deployment of the software. The Vite-built static bundle is deployed to Netlify's global CDN. Firebase Auth and Firestore are managed entirely through Google's Firebase Console with no self-hosted server components. Leaflet tiles are fetched from the OpenStreetMap CDN. GitHub serves as the source repository with automatic CI/CD pipelines triggering Netlify builds on each push to main.

7. Data Flow Diagram:

A Data Flow Diagram (DFD) visually represents the flow of data through the EcoPool system. It depicts how data enters (user form inputs, GPS coordinates, image uploads), moves through (Firestore writes, React state updates, onSnapshot callbacks), and is stored within the system (Firestore ride documents, user auth tokens), without detailing the timing or sequence of individual processes.

5.2.1. Use Case Diagram

The primary actors are Passenger and Driver (a single user can be both). Use cases include: Register/Login, Search Rides, Book Ride, View Route Map, Offer Ride, Manage Active Ride, Collect Payments, Rate Passengers, View Safety Info, Toggle Accessibility, and Use Voice Commands. The Firebase backend is an external system actor providing Auth and Firestore services.

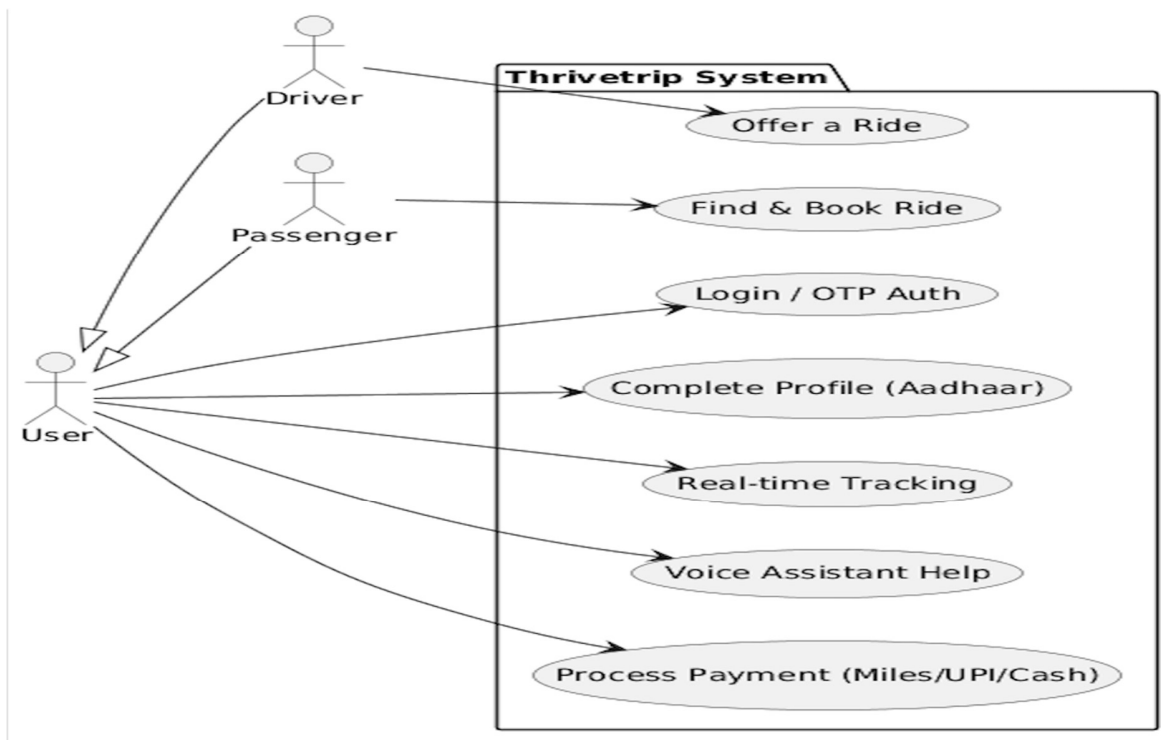


Figure 5.2 Use Case Diagram

5.2.2. Class Diagram

The EcoPool application architecture is built as a robust, client-side Single Page Application (SPA) that leverages a multi-provider state management pattern to ensure seamless data flow and high accessibility. At its core, the App component acts as the primary router, wrapping the entire system in an AuthContext to manage persistent user sessions and a metroMiles reward balance, as well as an AccessibilityContext that globally toggles a high-contrast mode for visually impaired users. The system's real-time capabilities are driven by the FindRide module, which utilizes Firestore onSnapshot listeners to instantly sync ride data between drivers and passengers, while the OfferRide component manages structured data entry for new trip postings. During the active phase of a commute, the RideActive component coordinates live GPS tracking via the useRideTracking hook and enforces a strict post-ride payment workflow through the DriverPaymentCollection and PaymentModal interfaces. This modular design is further enhanced by a hands-free useVoiceAssistant layer, ensuring that the platform remains safe and operational in high-traffic environments by allowing users to navigate and book rides using native browser speech APIs.

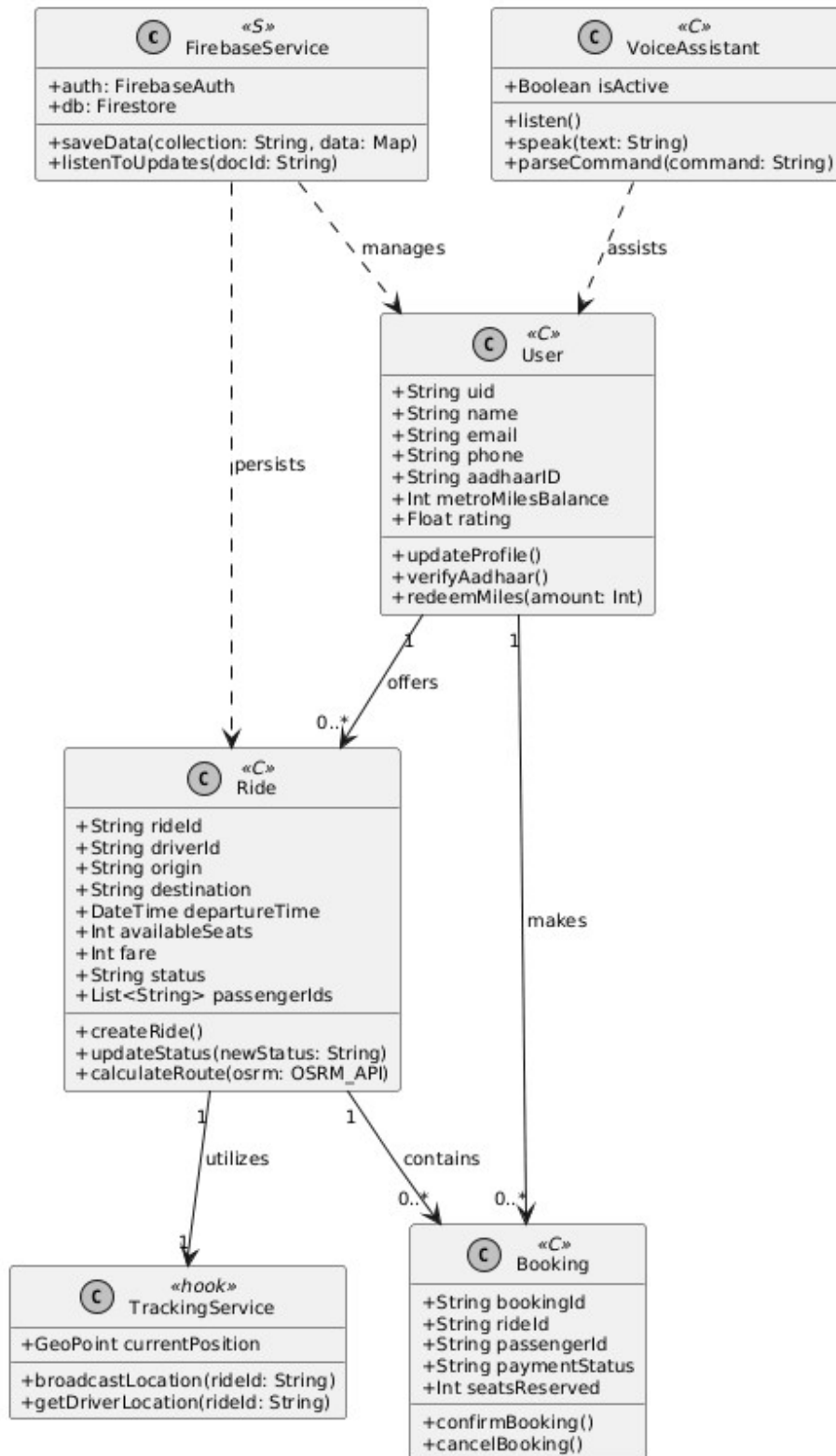


Figure 5.3 Class Diagram

5.2.3. Sequence Diagram

1. Routing and Initialization: When the user navigates to the FindRide page, the `useEffect` hook initializes the component and verifies authentication via the `AuthContext`.
2. Real-Time Data Binding: The `onSnapshot` listener creates a persistent connection to Cloud Firestore, ensuring ride updates are pushed instantly without a page refresh.
3. Reactive UI Rendering: The application maps over the Firestore collection to dynamically generate `RideCard` components that reflect real-time seat availability.
4. Geospatial Preview: Selecting a ride opens the `RouteMapModal`, which uses Leaflet and OpenStreetMap tiles to render a visual preview of the journey path.
5. User Commitment: The "Confirm Booking" action triggers a final validation check of the user's selected payment method or Metro Miles balance before the write operation.
6. Atomic Database Update: The `updateDoc` function decrements the available Seats and uses `array Union` to securely append the passenger's UID to the ride document.
7. Contextual GPS Snapshot: The system captures the passenger's current GPS coordinates via the browser API to establish a starting point for live tracking.
8. State Transition: Upon a successful write, the application uses React Router to transition the user to the Ride Active screen to manage the commute lifecycle.

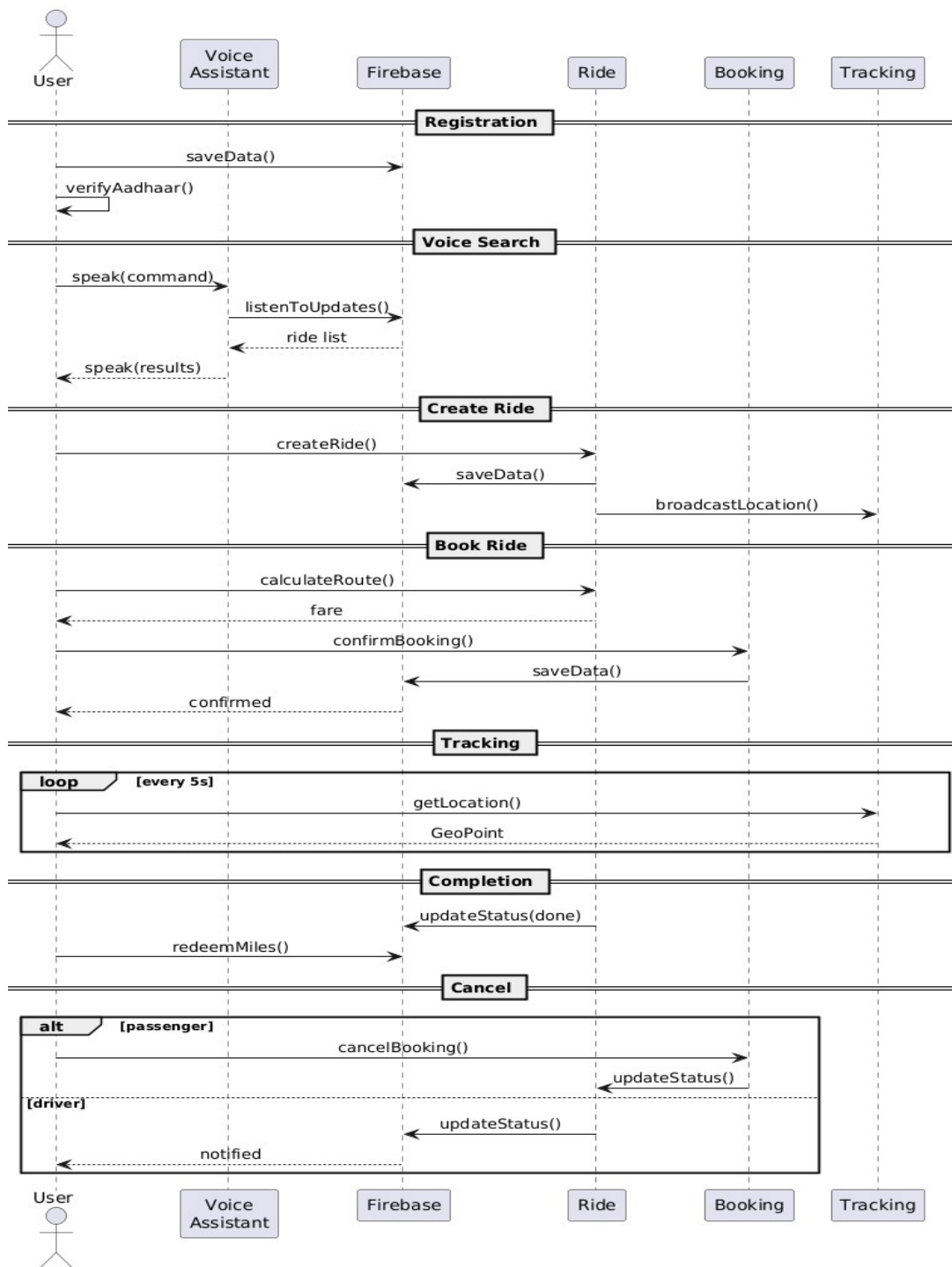


Figure 5.4 Sequence Diagram

5.2.4. Activity Diagram

The activity diagram models the dynamic flow of the application. It starts at the Login/Onboarding screen. On successful Firebase Auth, the user lands on FindRide. From the bottom navigation they can switch to OfferRide or Profile. Offering a ride leads to the OfferRide form, Firestore write, then redirect to FindRide. Booking a ride leads to RouteMapModal, then confirm, then Firestore update, then RideActive. On ride completion, the Payment modal appears; after all payments confirmed, the Rating flow triggers and the session resets.

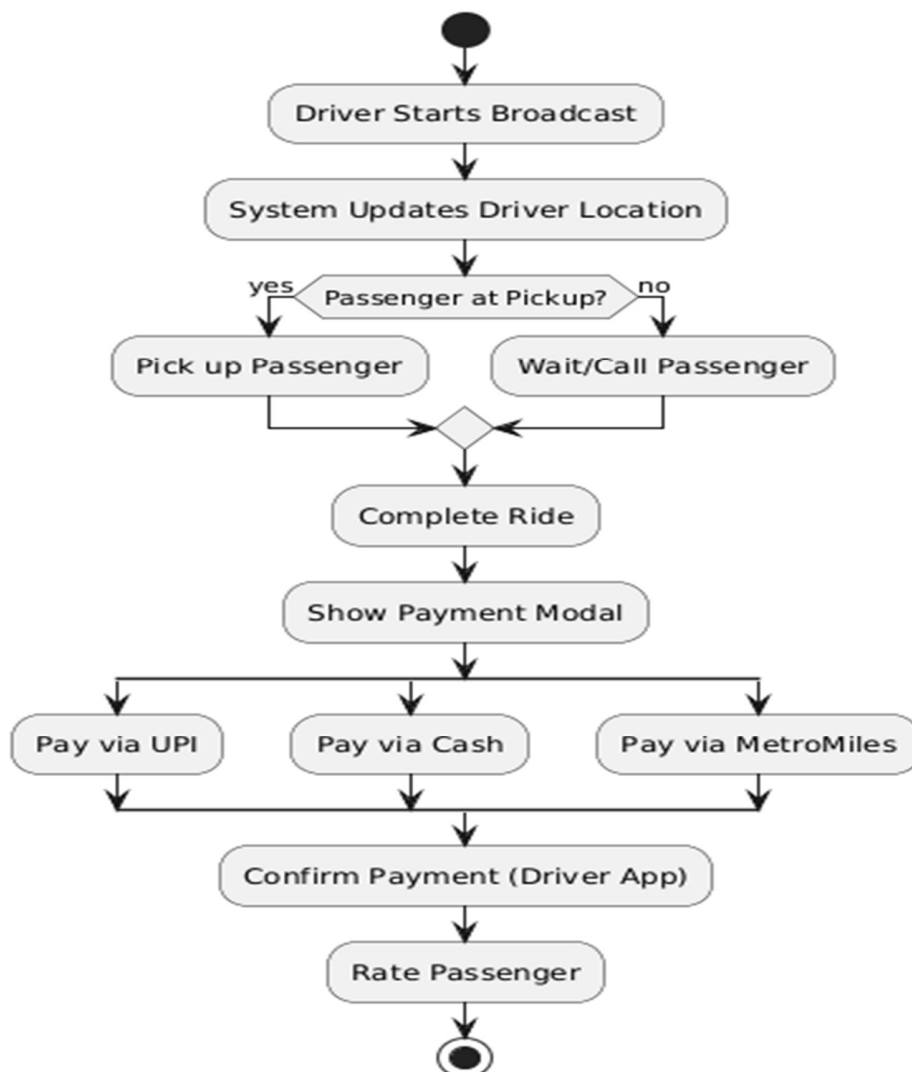


Figure 5.5 Activity Diagram

5.2.5. Component Diagram

The component diagram gives a high-level view of the system's physical components and their interactions. Key components include the Frontend React SPA (served by Netlify CDN), Firebase Authentication (JWT token issuance), Cloud Firestore (NoSQL ride document store), Leaflet Map Tiles (sourced from OpenStreetMap CDN), and the Web Speech API (browser-native speech recognition and synthesis). Each component is an independent module with well-defined interfaces, promoting modularity and ease of maintenance.

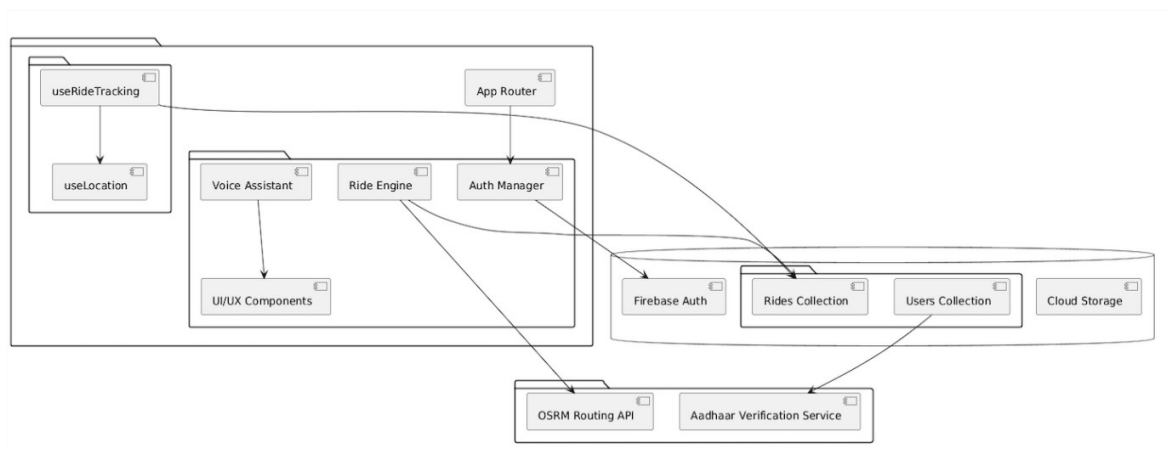


Figure 5.6 Component Diagram

5.2.6. Deployment Diagram

The deployment diagram outlines the physical deployment of EcoPool. The Vite-built static bundle (HTML, JS, CSS) is deployed to Netlify's global CDN with HTTPS enforced. Firebase Auth handles identity management server-side through Google's infrastructure. Cloud Firestore serves as the primary data store with automatic geo-redundancy. Leaflet tiles are served by the OpenStreetMap tile CDN. GitHub Actions (or Netlify's built-in CI) trigger fresh builds on every push to the main branch, ensuring continuous deployment with zero downtime.

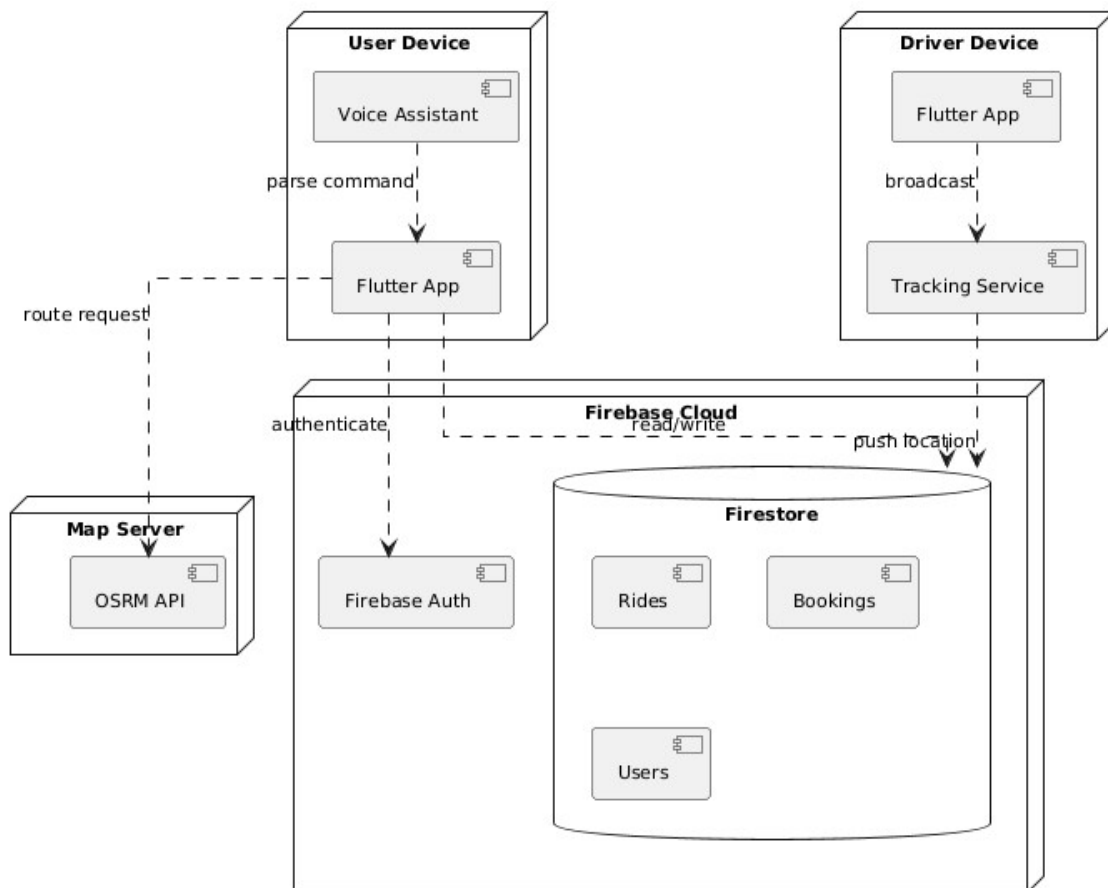


Figure 5.7 Deployment Diagram

5.3 Data Flow Diagrams

5.3.1 DFD Level 0

The Level 0 Data Flow Diagram (context diagram) provides a high-level overview of the Ride-Share System as a single process. It illustrates how the system interacts with its two external entities — the User and the Driver — along with the Firestore DB as the data store.

The User sends inputs such as ride booking, search, and cancellation requests to the system, and receives confirmation and status updates in return. The Driver interacts with the system by creating rides and broadcasting live location data, while receiving booking alerts as output. The Ride-Share System communicates with Firestore DB bidirectionally — storing ride, booking, and user data, and fetching it as needed to process requests. This diagram establishes the system boundary and defines all external data flows without detailing internal processes.

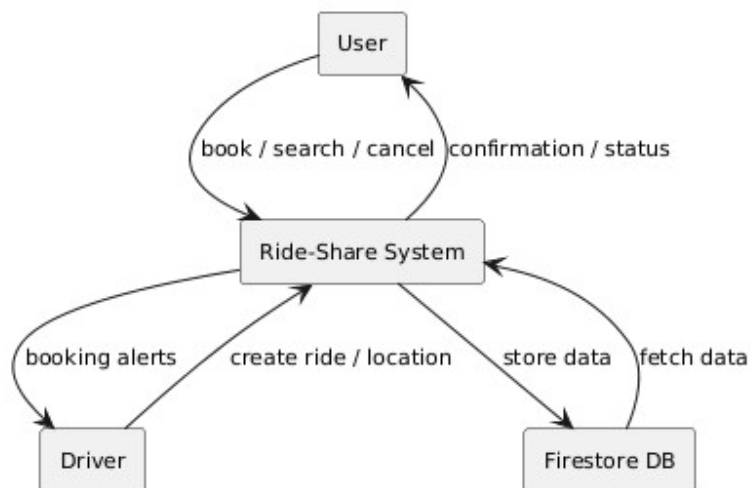


Figure 5.8 DFD level 0 diagram

5.3.2 DFD Level 1

The Level 1 DFD breaks the Ride-Share System into five internal processes. User Auth (1.0) verifies credentials against the Users DB and returns an auth token. The Voice Assistant (5.0) parses user commands and forwards queries to Ride Management (2.0), where the Driver creates rides saved to the Rides DB. Booking (3.0) handles ride confirmation, updating both the Bookings DB and Rides DB accordingly. Finally, Tracking (4.0) receives live location from the Driver and streams it to the User in real time.

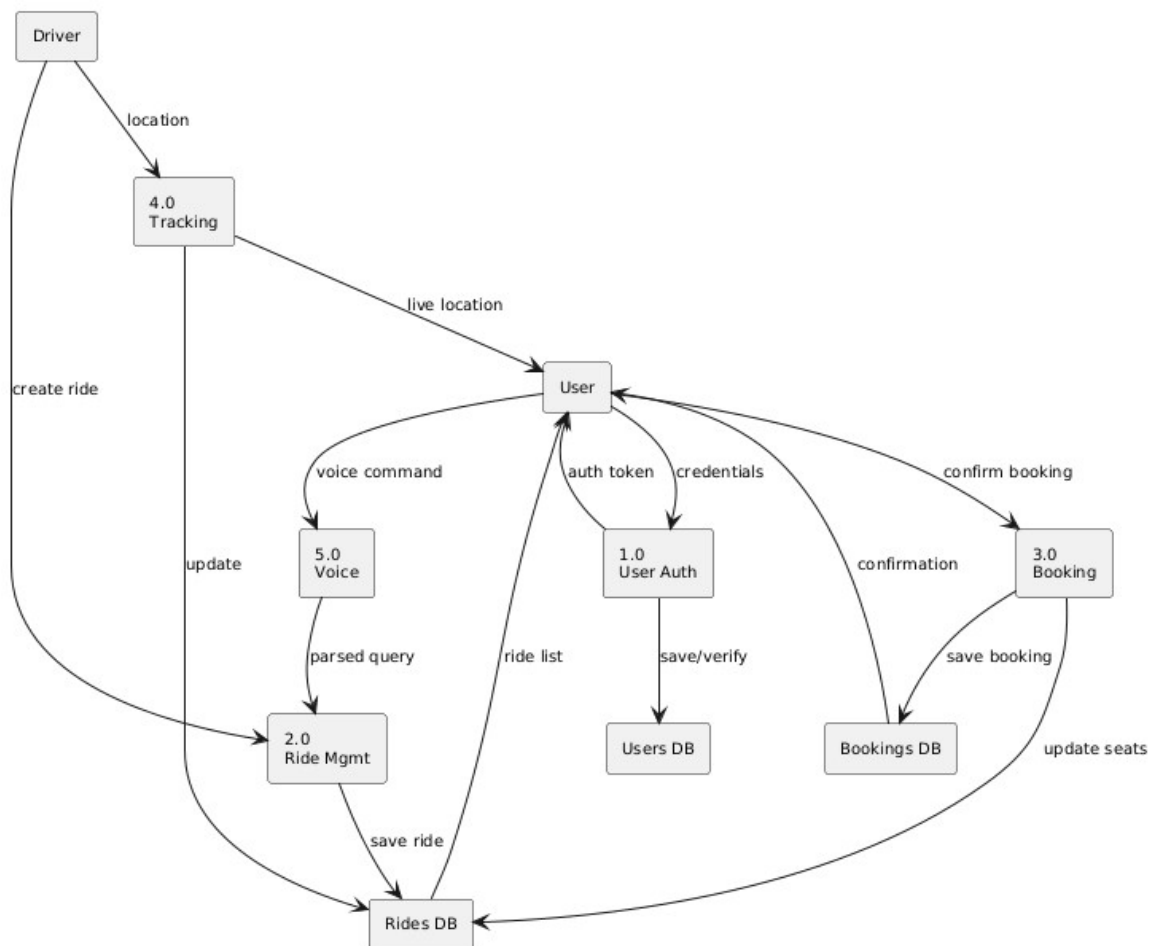


Figure 5.9 DFD level 1 diagram

6. IMPLEMENTATION

6.1. Software Development Cycle

An Agile-iterative SDLC was adopted with two-week sprints. Sprint 1 covered project scaffolding, Firebase project setup, and Authentication (AuthContext, Login, Register pages). Sprint 2 built the FindRide and OfferRide pages with full Firestore integration and real-time onSnapshot listeners. Sprint 3 added Leaflet maps, RouteMapModal, and the SmartSpotGuide accordion component. Sprint 4 implemented RideActive, PaymentModal, and DriverPaymentCollection with all three payment methods. Sprint 5 delivered the Safety, Settings, VoiceAssistant, and Accessibility features including the high-contrast toggle. Sprint 6 was dedicated to system testing, bug fixing, performance optimisation, and Netlify deployment.

6.2. Requirements Collection and Analysis

Requirements collection involved identifying the primary user journeys: a daily commuter searching for a ride to the Hitech City IT park, a driver offering a morning ride from JNTU Metro, and a student needing a voice-accessible booking experience. Both functional requirements (ride posting, booking, payment, safety) and non-functional requirements (sub-second updates, 99.95% availability, WCAG 2.1 AA compliance) were documented before any code was written. Firebase's capability matrix was reviewed to confirm free-tier quotas were sufficient for the expected load.

6.2.1 Authentication Module

AuthContext.jsx wraps the entire application with a Firebase onAuthStateChanged listener. It exposes currentUser, metroMiles (a mock balance initialized to 250 miles), login, register, and logout functions. Protected routes use a PrivateRoute wrapper component that redirects unauthenticated users to the Login page, preserving the intended destination URL for post-login redirect.

6.2.2 Ride Management Module

OfferRide.jsx collects 10 form fields and writes a structured ride document to Firestore with `serverTimestamp()`. FindRide.jsx attaches a real-time `onSnapshot` listener to the rides collection ordered by `createdAt` descending, updating the UI instantly when new rides are posted. A client-side filter function applies the `SearchBar` query against origin and destination fields without additional Firestore reads.

6.2.3 Map & Location Module

Map.jsx renders a Leaflet `MapContainer` centred on Hyderabad (17.3850°N, 78.4867°E) with coloured `CircleMarkers` for predefined hubs: JNTU Metro (red), Hitech City Mindspace (blue), and Secunderabad Station (green). `RouteMapModal.jsx` displays a full-screen Leaflet overlay for route preview before booking. `useRideTracking.js` uses the browser Geolocation API (`navigator.geolocation.watchPosition`) to save periodic location snapshots to Firestore at 30-second intervals during an active ride.

6.2.4 Payment Module

`PaymentModal.jsx` supports three tabs: Cash (instruction text), UPI (QR code icon rendered via `lucide-react`), and Miles (Metro Miles balance display from `AuthContext`). `DriverPaymentCollection.jsx` renders a per-passenger list with name, avatar initial, payment method badge, amount due, and a Confirm button. Confirmed payments show a green checkmark icon. The `Rate Passengers` button activates only when all passenger payments are confirmed, enforcing the payment-first workflow.

6.3 Sample Code

6.3.1 Offering a Ride (OfferRide.jsx – `handleSubmit`)

```
const handleSubmit = async (e) => {  
  e.preventDefault();  
  try {  
    await addDoc(collection(db, "rides"), {  
      origin: formData.origin,
```

```

    destination: formData.destination,
    date: formData.date,
    time: formData.time,
    pricePerSeat: Number(formData.pricePerSeat),
    availableSeats: Number(formData.availableSeats),
    vehicleModel: formData.vehicleModel,
    numberPlate: formData.numberPlate.toUpperCase(),
    vehicleColour: formData.vehicleColour,
    driverUid: currentUser.uid,
    passengers: [],
    createdAt: serverTimestamp()
  });
  navigate("/find-ride");
} catch (err) {
  setError("Failed to post ride: " + err.message);
}
};

```

6.3.2 Booking a Ride (FindRide.jsx – handleConfirmBook)

```

const handleConfirmBook = async () => {
  const rideRef = doc(db, "rides", selectedRide.id);
  await updateDoc(rideRef, {
    availableSeats: selectedRide.availableSeats - 1,
    passengers: arrayUnion({
      uid: currentUser.uid,
      name: currentUser.displayName || "Passenger",
      gps: await getCurrentPosition(),

```

```

        paymentMethod: "cash",
        paid: false
    })
  });
  setBookingConfirmed(true);
  navigate("/ride-active");
};

```

6.3.3 useRideTracking Hook (key excerpt)

```

useEffect(() => {
  if (!rideId || !currentUser) return;
  const watchId = navigator.geolocation.watchPosition(
    async (position) => {
      const { latitude, longitude } = position.coords;
      const rideRef = doc(db, "rides", rideId);
      await updateDoc(rideRef, {
        [`gps.${currentUser.uid}`]: { lat: latitude, lng:
longitude }
      });
    },
    (err) => console.error("Geolocation error:", err),
    { enableHighAccuracy: true, maximumAge: 30000 }
  );
  return () => navigator.geolocation.clearWatch(watchId);
}, [rideId, currentUser]);

```

6.3.4 main.jsx

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';

import { AuthProvider, useAuth } from '../context/AuthContext';

import { AccessibilityProvider } from '../context/AccessibilityContext';

import Layout from '../components/Layout';

import FindRide from '../pages/FindRide';

import OfferRide from '../pages/OfferRide';

import Profile from '../pages/Profile';

import Login from '../pages/Login';

import RideActive from '../pages/RideActive';

import Onboarding from '../pages/Onboarding';

import FAQ from '../pages/FAQ';

import Settings from '../pages/Settings';

import PaymentMethods from '../pages/PaymentMethods';

import Safety from '../pages/Safety';

import YourRides from '../pages/YourRides';

function PrivateRoute({ children }) {

  const { currentUser } = useAuth();

  return currentUser ? children : <Navigate to="/login" />;

}
```

```

function App() {
  return (
    <AccessibilityProvider>
      <AuthProvider>
        <BrowserRouter>
          <Routes>
            <Route path="/login" element={<Login />} />
            <Route path="/" element={
              <PrivateRoute>
                <Layout />
              </PrivateRoute>
            }>
            <Route index element={<FindRide />} />
            <Route path="offer" element={<OfferRide />} />
            <Route path="profile" element={<Profile />} />
            <Route path="active-ride/:rideId?" element={<RideActive />} />
            <Route path="onboarding" element={<Onboarding />} />
            <Route path="faq" element={<FAQ />} />
            <Route path="settings" element={<Settings />} />
            <Route path="payment/:rideId?" element={<PaymentMethods />} />
            <Route path="safety" element={<Safety />} />
            <Route path="your-rides" element={<YourRides />} />
          </Route>
        </Routes>
      </BrowserRouter>
    </AccessibilityProvider>
  )
}

```

```

    </AuthProvider>

    </AccessibilityProvider>

    );
}
export default App;

import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)

```

7. TESTING

Testing is a critical phase in the software development lifecycle, serving as the final gatekeeper before software is delivered to end users. It ensures that the product performs according to specifications, remains reliable in diverse conditions, and delivers accurate results. In the EcoPool project, thorough testing was conducted to validate the system's correctness, robustness, and efficiency. This chapter elaborates on the key types of testing employed, including system testing, module testing, integration testing, and acceptance testing.

7.1. System Testing

End-to-end system testing was performed by simulating full user journeys: register → login → search ride → book ride → ride active → payment → logout. Both happy-path and edge-case scenarios were covered, including no rides available, network timeout, invalid login credentials, booking a ride with only one seat remaining, and switching payment modes mid-collection. Production builds were tested on Netlify preview URLs with browser DevTools Network tab throttling simulating 3G, 4G, and WiFi conditions.

7.2. Module Testing

Individual React components were tested in isolation using browser DevTools and React Developer Tools extension. Props and state were inspected at each render cycle. Custom hooks (`useRideTracking`, `useVoiceAssistant`) were tested by mocking the Geolocation and SpeechRecognition APIs via browser console overrides. The `AuthContext` was tested by manually setting and clearing Firebase Auth state to verify protected route redirects fired correctly in all scenarios.

7.3. Integration Testing

Integration tests verified data flow between components and Firebase. Posting a ride via `OfferRide` and immediately observing the new `RideCard` appear in FindRide's live list confirmed the `onSnapshot` integration. Payment confirmation in `DriverPaymentCollection` was tested with all three payment methods (Cash, UPI, Metro Miles) across multiple simulated passenger profiles. The voice command "Book a ride" was tested against the

SpeechRecognition mock to confirm the first available RideCard preview opened and a spoken confirmation was synthesized.

7.4. Acceptance Testing

A group of five test users (including students and faculty members) performed acceptance testing on the deployed Netlify preview. Users were asked to complete three tasks: offer a ride from JNTU Metro to Hitech City, book an available ride, and use the voice command to navigate to the Safety page. Feedback highlighted the need for a loading spinner during booking (subsequently added) and clearer error messages for failed Firestore writes (also addressed). Overall usability rating: 4.2/5.

7.5. Test Cases

Test Case 1:

Feature	Input Value	Expected Output
Register	Valid email + password ≥ 6 chars	User created; redirected to Find Ride
Login	Correct credentials	Auth token issued; currentUser populated
Offer Ride	All fields, price=150, seats=3	Ride doc written to Firestore; redirect to FindRide
Find Ride Search	Query: "JNTU"	List filtered to rides with JNTU in origin/dest
Book Ride	Tap confirm on available ride	seats-1; passengerUID added; GPS saved
Payment Cash	Driver taps confirm on passenger	Status set to "paid"; green tick shown
High Contrast	Toggle button tapped	Yellow background; black text applied globally
Voice Command	"Book a ride"	First available ride preview opens; spoken confirmation

Table 7.1 – System Test Cases for EcoPool

8. SCREENSHOTS

The following screenshots illustrate the key screens and functionality of the EcoPool carpooling application as deployed on Netlify.

Figure 8.1 – Onboarding / Login Screen

The login page features the EcoPool branding with email and password input fields, a "Sign In" button in teal, and a "Register" link for new users. The radiant-glass background with a frosted card panel gives the application a modern aesthetic. Firebase Auth errors are displayed inline below the form fields, providing immediate feedback without page reloads.

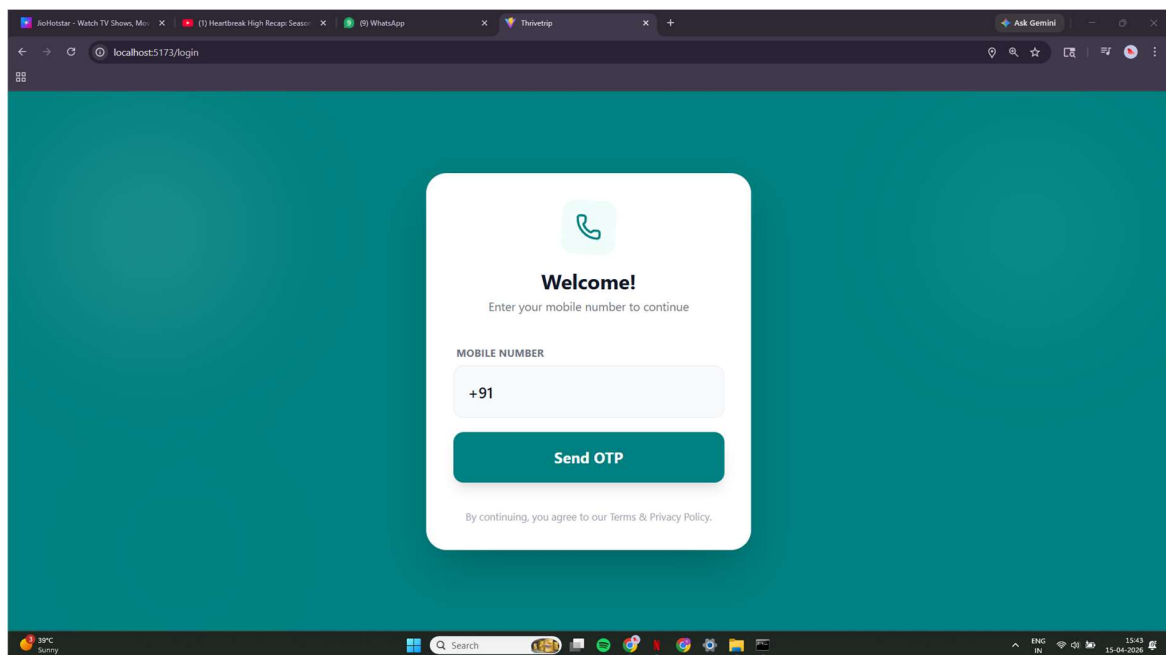


Figure 8.1 Onboarding / Login Screen

Figure 8.2 – Find Ride Page

The main screen shows the Leaflet map of Hyderabad with coloured hub markers (red for JNTU Metro, blue for Hitech City Mindspace, green for Secunderabad Station). Below the map is a SearchBar for filtering rides, a SmartSpotGuide accordion with popular pickup tips, and a scrollable list of RideCards showing origin, destination, price per seat, available seats, vehicle info, and a "Book" button.

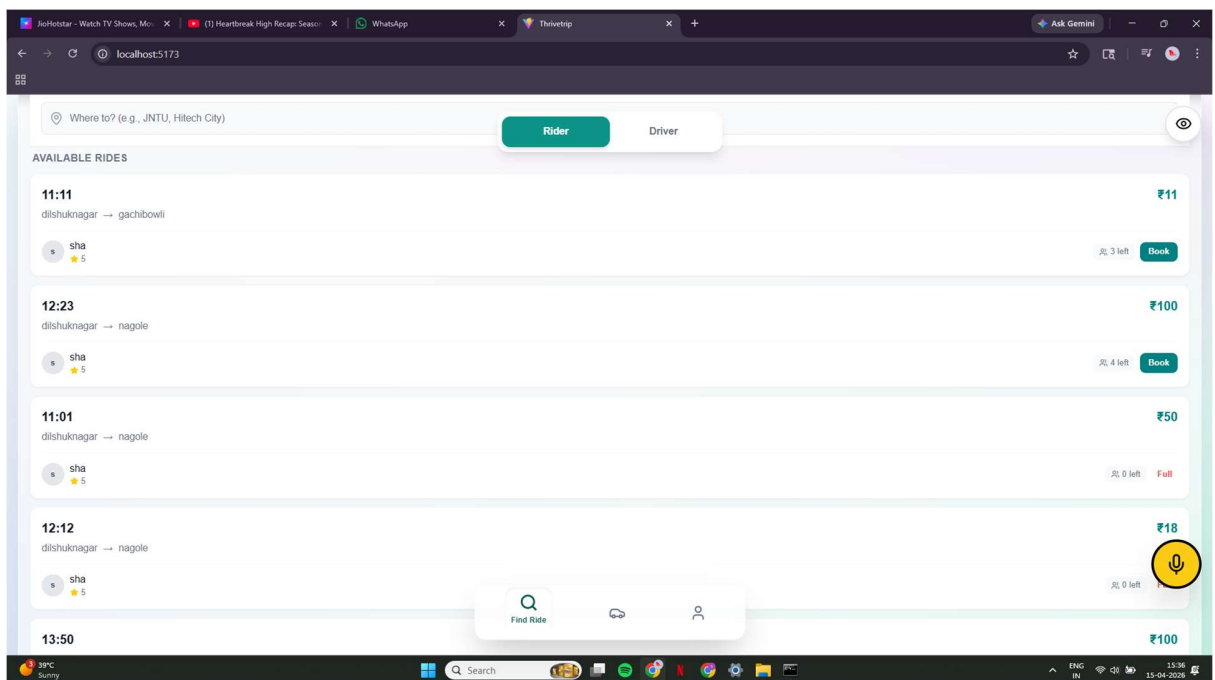
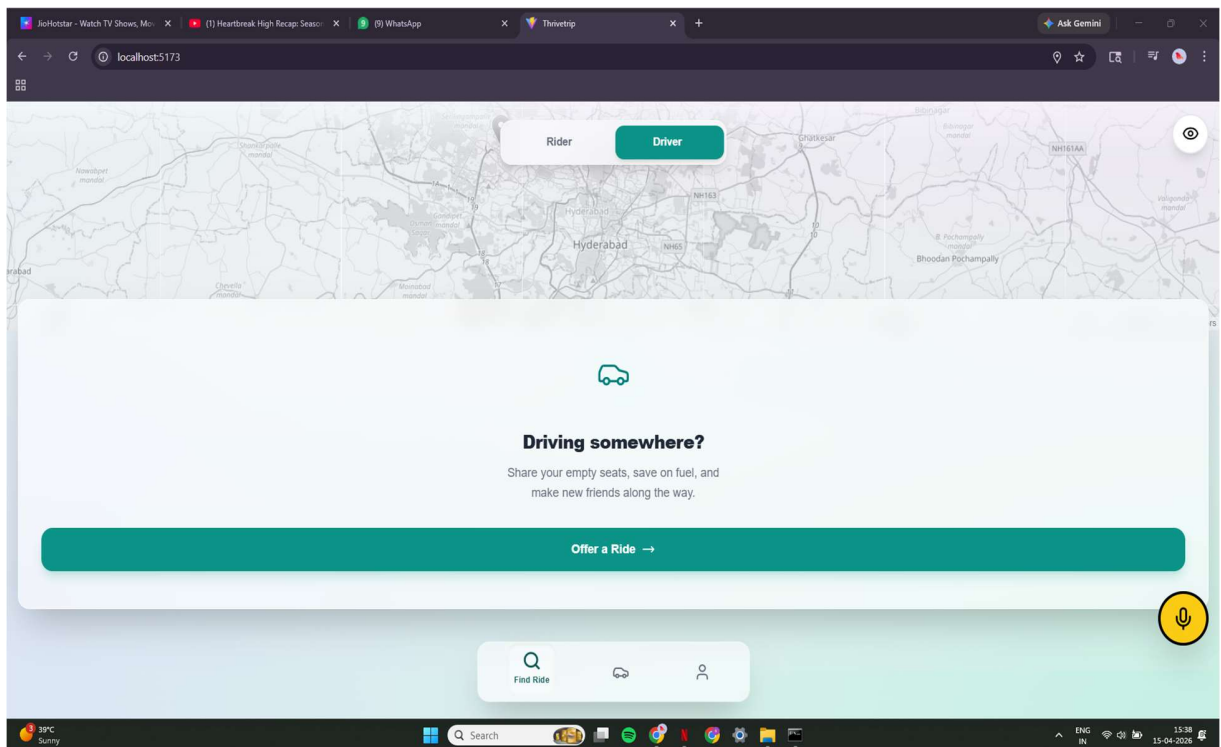


Figure 8.2 Find Ride Page

Figure 8.3 – Offer Ride Form

A clean form with 10 clearly labelled fields: Origin, Destination, Date, Time, Price/Seat, Available Seats, Vehicle Model, Number Plate (auto-uppercased on input), and Vehicle Colour. A teal "Offer Ride" submit button at the bottom triggers the Firestore addDoc write. Input validation prevents submission with empty required fields.



The image shows a web browser window with the address bar displaying 'localhost:5173/offer'. The page title is '+ Offer a Ride'. The form contains the following fields and elements:

- Origin:** A text input field with the placeholder 'e.g. My Home'.
- Destination:** A text input field with the placeholder 'e.g. Office, Mail, etc.'.
- Date:** A date picker field showing 'dd-mm-yyyy'.
- Time:** A time picker field showing '--:--'.
- Price (₹):** A text input field with the value '50'.
- Seats:** A text input field with the value '3'.
- Vehicle Details:** A section with a 'Car' option and a search icon.
- Buttons:** A yellow circular button with a microphone icon and a white button labeled 'Offer Ride'.

Figure 8.3 Offer Ride Form

Figure 8.4 – Route Map Modal

A full-screen map overlay appears when a user selects a ride to preview. It displays the pick-up and drop-off hub markers with a dashed route line. A "Confirm Booking" button at the bottom triggers the Firestore updateDoc booking transaction, decrementing available seats and appending the passenger to the ride document.

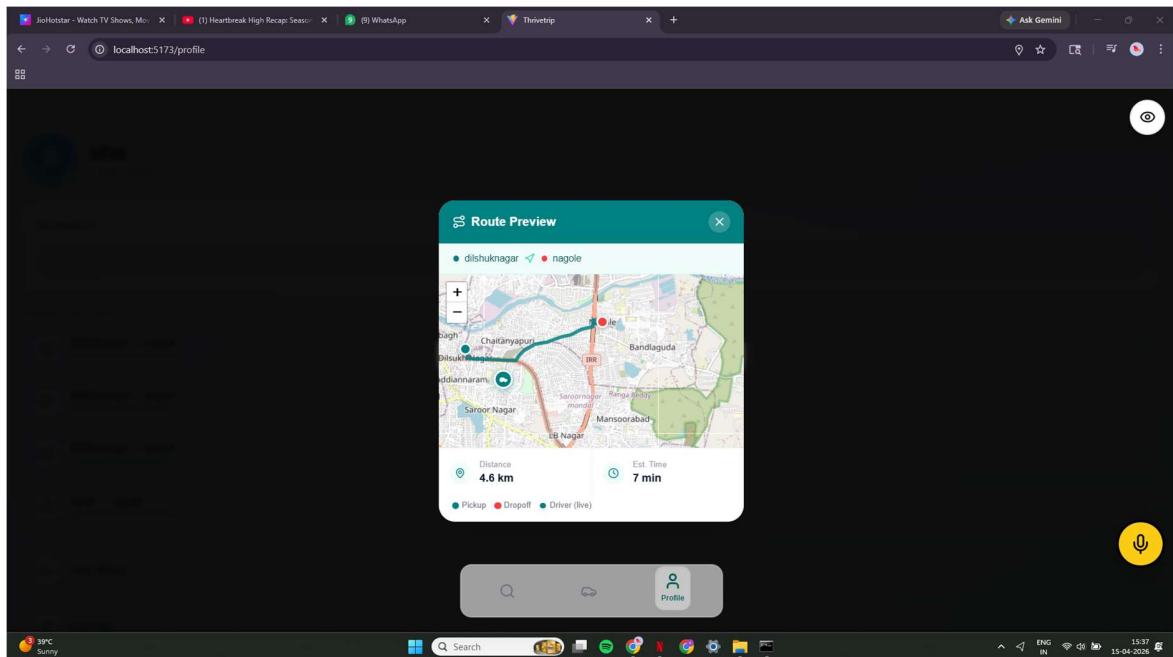


Figure 8.4 Route Map Modal

Figure 8.5 – Payment Collection Screen (Driver)

The DriverPaymentCollection modal lists each passenger with their display name, avatar initial, payment method badge (Cash / UPI / Miles), amount due, and a "Confirm" button. Confirmed payments show a green checkmark. The "Rate Passengers" button is disabled until all passenger payments are individually confirmed, enforcing the complete payment workflow.

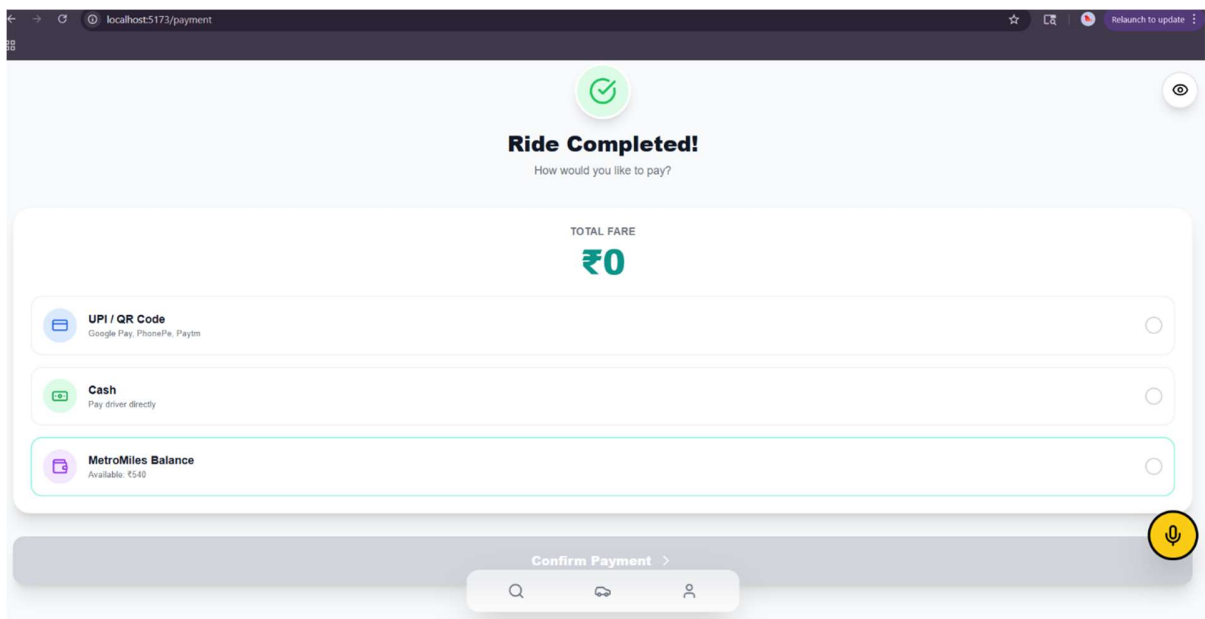


Figure 8.5 Payment Collection Screen

Figure 8.6 – Safety & Privacy Page

A dedicated safety page with an emergency assistance card featuring a prominent SOS button, a privacy toggles section for Location Sharing, Profile Visibility, and Data Sharing, and a community safety guidelines accordion. The teal header bar contains a back arrow for navigation. All toggle states are persisted in React local state and can be connected to Firestore user preferences in future versions.

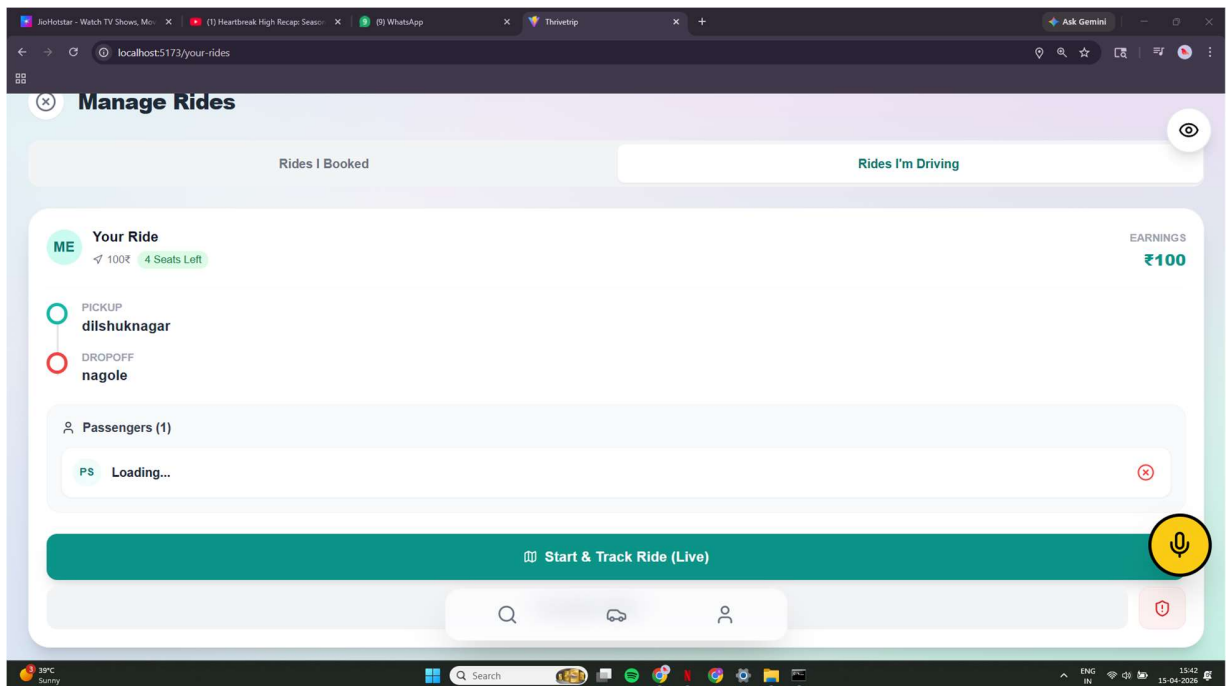


Figure 8.6 Safety & Privacy Page

Figure 8.7 – Active Ride Tracking Screen

The RideActive screen displays the ongoing ride details including driver information, current route status, and live location indicator. The useRideTracking hook continuously updates GPS coordinates to Firestore, allowing both driver and passengers to track the ride's progress. A prominent "End Ride" button triggers the payment collection workflow upon completion of the journey.

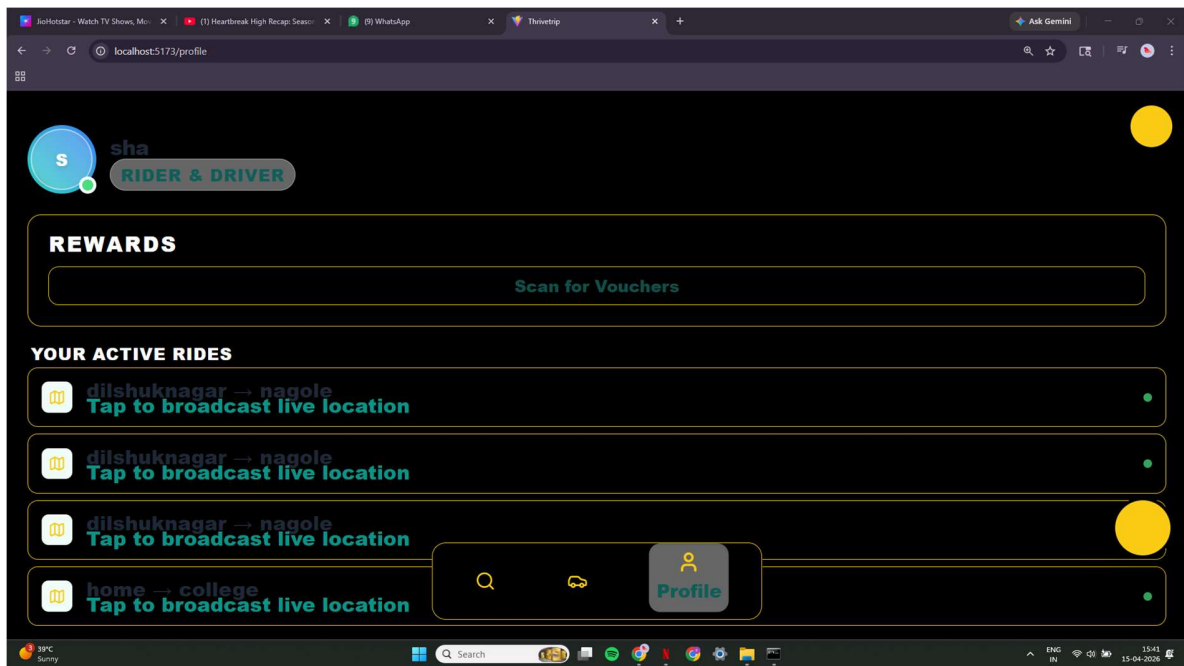


Figure 8.7 Active Ride Tracking Screen

9. CONCLUSION

The development of EcoPool — a modern carpooling platform — demonstrates the power of contemporary web technologies in solving real-world urban mobility challenges. By combining React 18, Firebase, Leaflet, and Tailwind CSS, the team delivered a production-quality single-page application that supports end-to-end ride management: from offering and searching rides, to real-time booking, live location tracking, multi-modal payment, and safety features — all without requiring a native mobile app installation.

The project's modular component architecture, context-based state management, and custom hooks (`useRideTracking`, `useVoiceAssistant`) provide a scalable foundation for future feature development. The Agile development methodology, combined with continuous testing at unit, integration, and acceptance levels, ensured that the final product is both functionally correct and user-friendly.

Accessibility was a first-class concern throughout: the high-contrast toggle and voice assistant make EcoPool usable by a wider audience than most commercial carpooling applications. The Metro Miles barter payment system introduces a novel, community-centric economic model that could encourage adoption in college campuses and residential communities where monetary transactions may be inconvenient.

Looking ahead, EcoPool can be enhanced with AI-driven ride matching (optimising route overlap via graph algorithms), in-app real-time messaging between driver and passengers, push notifications via Firebase Cloud Messaging, integration with TSRTC and Hyderabad Metro real-time APIs for multimodal journey planning, and a comprehensive driver and passenger rating system backed by Firestore aggregation queries. Deployment to production at scale would benefit from migrating to Firebase's Blaze plan and adding Cloud Functions for server-side business logic such as automated refunds and fraud detection.

In conclusion, EcoPool not only fulfils its intended purpose as a university project but also represents a genuinely deployable product with clear social value. It stands as evidence that thoughtful application of modern open-source technologies can yield professional-grade solutions to meaningful community problems. The project lays a strong foundation for future contributions to sustainable urban mobility in Hyderabad and beyond.

10. REFERENCES

- [1] React Documentation. "React 18 – Getting Started." <https://react.dev>, 2022.
- [2] Firebase. "Firebase Documentation – Firestore & Authentication." <https://firebase.google.com/docs>, 2024.
- [3] Vite. "Vite – Next Generation Frontend Tooling." <https://vitejs.dev>, 2023.
- [4] Tailwind CSS. "Utility-First CSS Framework Documentation." <https://tailwindcss.com/docs>, 2023.
- [5] Leaflet. "Leaflet – an open-source JavaScript library for mobile-friendly interactive maps." <https://leafletjs.com>, 2023.
- [6] react-leaflet. "React components for Leaflet maps." <https://react-leaflet.js.org>, 2023.
- [7] Aggarwal, A. & Singh, P. "Carpooling Solutions for Smart Cities: A Review." *International Journal of Intelligent Transportation Systems Research*, 2021.
- [8] Mehta, R. "Building Real-Time Web Apps with Firebase and React." *Proceedings of the Web Developer Conference*, 2021.
- [9] Nguyen, T. & Kim, R. "Progressive Web Applications vs. Native Apps for Mobility Services." *ACM SIGCHI Conference Companion*, 2022.
- [10] Patel, J. & Chen, M. "Scalability Analysis of Firebase Firestore for Transportation Applications." *IEEE Cloud Computing Symposium*, 2022.
- [11] Kapoor, S. & Rao, D. "Accessible Interface Design for Mobility Platforms." *Accessibility and HCI Journal*, 2023.
- [12] MDN Web Docs. "Web Speech API." https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API, 2023.
- [13] OpenStreetMap Contributors. "OpenStreetMap." <https://www.openstreetmap.org>, 2024.